ГКП НА ПХВ «ВЫСШИЙ КОЛЛЕДЖ «ASTANA POLYTECHNIC»

АКИМАТА Г. АСТАНА

КУРСОВОЙ ПРОЕКТ

(Пояснительная записка)

Разработка серверной части

интерактивного каталога персонажей ICC

КП.06130100.ИС-24-22б.01.11.26.ПЗ

Выполнил Ибрагимов Р. Р.

(дата, подпись)

Руководитель Попов Д. В.

(дата, подпись)

2026г.

ЗАДАНИЕ НА КУРСОВОЙ ПРОЕКТ

Дисциплина: Основы backend-разработки. Специальность: 06130100 Программное обеспечение. Группа: ИС-24-22б.

Обучающийся: Ибрагимов Ратмир Рустамович. Руководитель: Попов Д. В.

Тема: разработка серверной части интерактивного каталога персонажей ICC.

Исходные данные: техническое задание ICC и концептуальная модель КП 2; условия КП 3–12 и ПР 13 курса «ИС-24-22б Основы backend».

Разработать REST API на PHP и Laravel с MySQL 8.4, токенами Sanctum, разграничением прав, хранением PNG и проверкой сценария модерации мейкера. Подготовить миграции, модели, начальные данные, спецификацию API, тесты и инструкцию запуска в OSPanel и VS Code.

Содержание работ

Этап	**Результат**
Анализ	Предметная область, аналоги, роли, требования и варианты использования
Проектирование	Архитектура сервера, модель MySQL, API и механизмы безопасности
Реализация	Маршруты, валидация, сервисы, Eloquent, файлы и модерация
Проверка	Автотесты MySQL, запросы API, контроль транзакций и документация
Оформление	Пояснительная записка DOCX и PDF; исходный код в GitHub

Срок представления по Classroom: 20 ноября 2026 года. Результат: серверный проект и пояснительная записка с приложениями.

Дата выдачи _____ Подпись руководителя _____

Задание принял _____ Подпись обучающегося _____

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4
1 АНАЛИТИКО-ПОСТАНОВОЧНАЯ ЧАСТЬ 6
1.1 Описание предметной области 6
1.2 Анализ процесса и выявленных проблем 6
1.3 Анализ существующих аналогов 7
1.4 Роли и варианты использования 8
1.5 Функциональные и нефункциональные требования 10
1.6 Ограничения и выводы по разделу 11
2 ТЕОРЕТИКО-ПРОЕКТНАЯ ЧАСТЬ 12
2.1 Общая архитектура серверной части 12
2.2 Выбор и обоснование технологий 12
2.3 Проектирование базы данных 14
2.4 Слои приложения и ключевые сценарии 21
2.5 Проектирование API 23
2.6 Проектирование механизмов безопасности 24
2.7 Контроль версий и выводы по разделу 25
3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И ОЦЕНКА РЕЗУЛЬТАТОВ 26
3.1 Среда разработки и история коммитов 26
3.2 Реализация базы данных и начальное наполнение 28
3.3 Маршруты валидация ответы и ошибки 29
3.4 Аутентификация авторизация и файлы 30
3.5 Бизнес логика транзакции и списки 31
3.6 Журналирование и документирование API 33
3.7 Тестирование серверной части 33
3.8 Развёртывание в OSPanel и оценка результатов 37
ЗАКЛЮЧЕНИЕ 40
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 41
ПРИЛОЖЕНИЕ 1 СПЕЦИФИКАЦИЯ API 43
ПРИЛОЖЕНИЕ 2 МИГРАЦИИ БАЗЫ ДАННЫХ 47
ПРИЛОЖЕНИЕ 3 ОСНОВНЫЕ СЕРВЕРНЫЕ МОДУЛИ 50
ПРИЛОЖЕНИЕ 4 РЕПОЗИТОРИЙ И МАТЕРИАЛЫ ПРОВЕРКИ 54

ВВЕДЕНИЕ

ICC (Interactive Character Catalog) предназначен для хранения наборов графических элементов, конфигураций персонажей и пользовательских публикаций. Для такого каталога недостаточно сохранить PNG в общей папке: необходимо знать автора каждого набора, отделять черновики от одобренных материалов и проверять совместимость выбранных элементов. Без единого серверного контроля пользователь может обратиться к чужому объекту по его идентификатору, сохранить несовместимую конфигурацию или повторно опубликовать изменённый набор без проверки.

В учебной модели ICC автоматизируется процесс подготовки и модерации мейкера. Креатор передаёт набор слоёв, администратор принимает решение, после одобрения данные становятся доступны через публичный API. Сервер хранит состояние процесса в MySQL и применяет одинаковые правила к каждому запросу. Это позволяет проверять сценарий независимо от интерфейса и повторять его на демонстрационных данных [6, 5].

Цель курсового проекта — разработать серверную часть ICC с REST API, реляционной базой MySQL, разграничением доступа и транзакционной обработкой модерации, обеспечивающую сохранение совместимых персонажей и работу с публикациями.

Для достижения цели поставлены следующие задачи:

1) Проанализировать предметную область, сопоставить аналоги и определить проверяемые требования к серверу.

2) Спроектировать архитектуру приложения, нормализованную модель данных и контракт API.

3) Реализовать схему MySQL миграциями, модели Eloquent, фабрики и начальное наполнение.

4) Реализовать маршрутизацию, валидацию, формирование JSON и единообразную обработку ошибок.

5) Настроить Sanctum, роли и политики владения объектами, защиту файлов и журналирование.

6) Реализовать модерацию, транзакции, поиск, фильтры, сортировку и пагинацию.

7) Проверить сервер на MySQL, подготовить документацию и воспроизводимый запуск в OSPanel.

Объект разработки — процесс хранения и публикации наборов графических элементов и конфигураций персонажей. Предмет разработки — серверная модель данных ICC, REST API, правила доступа и обработки заявок на модерацию.

Средства реализации: PHP 8.4, Laravel 13, Eloquent ORM (объектно-реляционное отображение), Laravel Sanctum и MySQL 8.4. Работа выполняется в Windows с OSPanel и VS Code. Для контроля используются Git, функциональные тесты Laravel и воспроизводимая Postman-коллекция. Файлы изображений располагаются отдельно от реляционных данных.

Практическая значимость результата заключается в возможности выдавать публичные материалы и сохранять личные данные через единый проверяемый интерфейс. Администратор получает управляемый процесс рассмотрения заявки; креатор видит результат проверки; пользователь работает только со своими сохранёнными персонажами. Состояние не зависит от того, каким клиентом был отправлен запрос.

Проект ICC указан в исходном техническом задании как совместная работа Недосековой А. И. и Ибрагимова Р. Р. Данная записка представляет серверный результат Ибрагимова Р. Р.: базу данных, API, сервисы, безопасность, тестирование и развёртывание. Репозиторий проекта: https://github.com/Rat-mir/ICC.

Записка состоит из трёх разделов. Первый определяет проблему и требования. Во втором обоснованы технологии, архитектура, структура БД и контракт API. В третьем приведены реализация, протокол проверки и инструкция запуска. Приложения содержат спецификацию маршрутов, миграции и основные серверные листинги.

1 АНАЛИТИКО-ПОСТАНОВОЧНАЯ ЧАСТЬ

1.1 Описание предметной области

Рассматривается учебная платформа, имитирующая каталог авторских конструкторов персонажей. Мейкер объединяет PNG-элементы определённых категорий. Каждый элемент принадлежит одному набору, а сохранённый персонаж содержит выбор элементов и их порядок. Публикация хранит готовое изображение и может ссылаться на сохранённого персонажа. Комментарии и лайки относятся к публикации [6, 5].

Входные данные сервера — регистрационные сведения, параметры мейкера, PNG-файлы, список выбранных элементов, текст публикации, комментарии и решение модератора. Выходные данные — JSON с ресурсами, списками и ошибками, а также PNG после проверки доступа. Основные термины представлены в таблице 1.1.

Таблица 1.1 Термины предметной области

Термин	**Смысл в ICC**
Мейкер Maker	Авторский набор графических элементов со статусом модерации
Элемент Asset	PNG-файл, категория и порядок слоя в наборе
Персонаж Character	Сохранённая конфигурация выбора элементов одного мейкера
Публикация Post	Готовый PNG с названием и владельцем
Креатор	Пользователь с правом создавать и передавать мейкеры на проверку
Модерация	Проверка наполнения и принятие решения администратором
Приватный ресурс	Материал, доступный владельцу и администратору

Демонстрационное наполнение включает 10 аккаунтов, 10 мейкеров, 60 элементов, 10 персонажей, 10 публикаций, 10 комментариев и 10 лайков. Эти количества описывают исходные тестовые данные, а не реальную аудиторию. После выполнения сценариев в рабочей базе могут находиться дополнительные записи.

1.2 Анализ процесса и выявленных проблем

До автоматизации рассматривается модельный процесс передачи PNG и замечаний отдельными файлами и сообщениями. Автор вручную сообщает, какой набор готов к публикации; проверяющий отправляет замечания; затем автор собирает новую версию. При таком процессе нет единого источника сведений о текущем статусе. Имя файла не гарантирует ни принадлежность элемента, ни совместимость его размеров с остальным набором.

Для серверной части существенны четыре проблемы: разрозненное хранение метаданных, отсутствие проверки связей, неконтролируемая смена статуса и возможность обращения к чужим данным. Требуется хранить связи и права централизованно, а изменение публикационной доступности выполнять в одной операции. Соответствие проблем проектным решениям приведено в таблице 1.2.

Таблица 1.2 Проблемы исходного процесса

Участник	**Исходное действие**	**Проблема**	**Изменение**
Креатор	Передаёт папку PNG	Теряются категории и порядок	Asset связан с Maker, категории проверяются
Модератор	Отправляет замечание сообщением	Неясен текущий статус	Статус и последнее решение хранятся в БД
Пользователь	Сохраняет набор идентификаторов	Возможны чужие и неактивные элементы	Сервис проверяет состав и принадлежность
Администратор	Вручную считает материалы	Сводка быстро устаревает	API агрегирует данные MySQL

Целевой процесс строится вокруг идентификатора мейкера. Черновик, файлы и решение связаны с одной записью; повторная отправка требует допустимого статуса. Публичный запрос фильтрует данные на сервере. Такое решение устраняет зависимость доступа от скрытых кнопок или пользовательских настроек запроса.

1.3 Анализ существующих аналогов

Для сравнения выбраны Picrew, CHARAT и meiker.io. Их открытые официальные страницы подтверждают назначение сервисов, но не раскрывают внутреннюю структуру БД и реализацию авторизации. Поэтому сравниваются документированные возможности и применимость к учебному серверному проекту, без предположений об устройстве чужого бэкенда [16, 2, 13].

Таблица 1.3 Сопоставление аналогов

Критерий	**Picrew**	**CHARAT**	**meiker.io**	**ICC**
Основная задача	Каталог image makers	Создание аватаров	Конструкторы dress up	Каталог и конфигурации
Создание авторского набора	Роль creator	Готовые конструкторы	Загрузка PSD	Загрузка PNG по API
Группы графических частей	Части мейкера	Варианты образа	Слои PSD	6 категорий Asset
Исходный формат автора	Части изображений	Зависит от конструктора	PSD	PNG до 5 МБ
Публичный каталог	Да	Подборка конструкторов	Да	Только approved
Учебная модель MySQL	Не раскрыта	Не раскрыта	Не раскрыта	Миграции и DDL
Документированный API ICC	Не относится	Не относится	Не относится	37 маршрутов
Локальное развёртывание ICC	Не относится	Не относится	Не относится	OSPanel и Artisan

Picrew показывает значимость самостоятельной роли автора, CHARAT — востребованность выбора готовых частей, meiker.io — возможность превращать авторские слои в конструктор. ICC использует идею каталога совместимых элементов, однако решает учебную задачу собственного проверяемого API на Laravel и MySQL. Использование внешнего сервиса не заменяет требуемые миграции, сервисы, политики и тесты.

1.4 Роли и варианты использования

Гость не является значением поля role: это запрос без установленного пользователя. В базе определены роли user, creator и admin. Креатор сохраняет возможности обычного пользователя. Администратор имеет доступ к административным маршрутам и объектам, однако бизнес-правила допустимых переходов сохраняются и для него. Матрица доступа приведена в таблице 1.4.

Таблица 1.4 Матрица серверных прав

Операция	**Гость**	**user**	**creator**	**admin**
Чтение approved и видимых постов	Да	Да	Да	Да
Личные персонажи	Нет	Свои	Свои	Все
Создание публикации и реакции	Нет	Да	Да	Да
Удаление публикации и комментария	Нет	Свои	Свои	Все
Создание мейкера и PNG	Нет	Нет	Свои	Да
Передача на модерацию	Нет	Нет	Свои	Да
Решение и скрытие	Нет	Нет	Нет	Да
Роли и блокировка аккаунтов	Нет	Нет	Нет	Да

На рисунке 1.1 показаны варианты использования именно серверного API. Связь с актором означает разрешённый запрос; фактическая доступность конкретного объекта дополнительно зависит от владельца и статуса. Регистрация и вход представлены отдельным вариантом для посетителя.

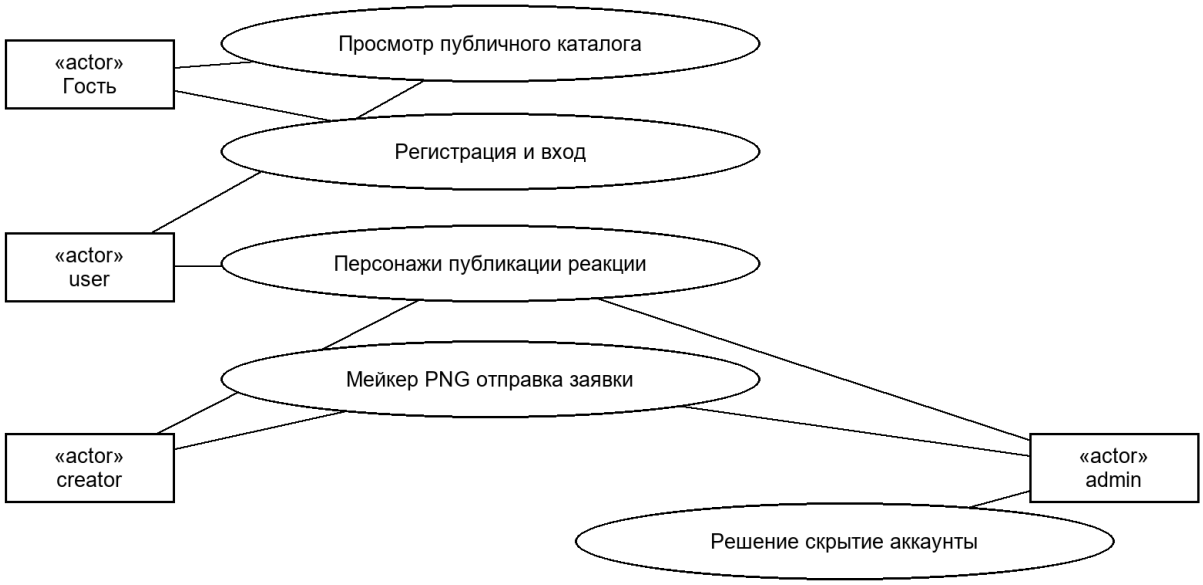


Рисунок 1.1 Варианты использования серверного API

Проверка роли выполняется до изменения данных. При корректной роли политика проверяет владение объектом; затем сервис проверяет его состояние. Поэтому право креатора работать с мейкерами не означает право редактировать чужую запись или заявку pending.

Таблица 1.5 Пользовательские истории и приёмка

История	**Критерий приёмки**
Креатор передаёт набор на проверку	При body и face одинакового размера draft становится pending
Администратор возвращает набор	Без причины ответ 422; с причиной status=revision
Пользователь сохраняет персонажа	Все элементы одного approved-мейкера, порядок хранится в pivot
Гость получает публичные материалы	В ответе нет draft, pending, revision, rejected и hidden

1.5 Функциональные и нефункциональные требования

Требования основаны на техническом задании и ограничены серверной ответственностью. Выбор частей и экспорт в браузере из исходного ТЗ обеспечиваются выдачей разрешённых PNG и метаданных. В серверной части проверяется сохранение состава, а не рисование изображения пользователем [6]. Проверяемые требования перечислены в таблицах 1.6 и 1.7.

Таблица 1.6 Функциональные требования

Код	**Проверяемая функция**
ФТ1	Выдавать гостю только одобренные мейкеры и видимые публикации
ФТ2	Регистрировать уникальный email, выдавать и отзывать токены
ФТ3	Искать и фильтровать мейкеры с сортировкой и пагинацией
ФТ4	Сохранять и изменять совместимые конфигурации своих персонажей
ФТ5	Создавать публикации с PNG и удалять свои публикации
ФТ6	Добавлять и удалять комментарии, ставить и снимать один лайк
ФТ7	Создавать мейкеры, хранить категории и порядок PNG-элементов
ФТ8	Отправлять draft или revision на модерацию после проверки набора
ФТ9	Принимать решение admin с причиной для возврата и отказа
ФТ10	Скрывать мейкеры и посты, управлять ролью и блокировкой
ФТ11	Повторно проверять исправленный или изменённый approved-набор
ФТ12	Возвращать административную агрегированную статистику

Таблица 1.7 Нефункциональные требования

Код	Критерий
НФТ1	PHP Laravel REST API, JSON, MySQL 8.4 и миграции
НФТ2	Sanctum Bearer; хеши паролей; контроль роли и владельца
НФТ3	Ошибки 401 403 404 409 422 в JSON; 500 без SQL и стека
НФТ4	Только PNG до 5 МБ, размер стороны от 16 до 2048 пикселей
НФТ5	Пагинация по умолчанию 20, разрешённый per_page от 1 до 100
НФТ6	Транзакции и блокировка конкурентного изменения мейкера
НФТ7	Журнал метода пути статуса пользователя и длительности без секретов
НФТ8	HTTPS в OSPanel, CORS только для заданных origin
НФТ9	Документация маршрутов; воспроизводимые тесты на отдельной MySQL

Число поддерживаемых пользователей и гарантированное время ответа не задаются: нагрузочные испытания не проводились. Функциональный протокол показывает корректность проверенных операций, но не заменяет оценку ёмкости системы. Для каждого требования в подразделе 3.8 приведено доказательство.

1.6 Ограничения и выводы по разделу

Проект развёрнут на локальном учебном стенде OSPanel. Используются демонстрационные аккаунты и процедурно созданные PNG. MySQL является единственной СУБД рабочей и тестовой конфигурации. Сервер не предоставляет рассылку, внешние платежи, автоматическую оценку запрещённого содержания или историю всех решений модератора. Последнее решение хранится в Maker, как предусмотрено минимальной моделью КП 2 [5].

Блокировка аккаунта и назначение роли доступны администратору. Интерфейс API не предоставляет самостоятельного повышения роли при регистрации. Окончательная проверка содержания выполняется человеком; сервер проверяет технические параметры, целостность и права. Развёртывание в интернете, резервирование оборудования и нагрузочное тестирование относятся к дальнейшему развитию.

Выявленная проблема состоит в отсутствии единого контроля состояния набора и доступа к связанным материалам. Для её решения необходимы реляционные связи, проверка совместимости, роли и транзакционные переходы модерации. Требования раздела задают конкретные наблюдаемые ответы API и ограничения БД. Эти критерии определяют архитектуру и программу проверки сервера.

2 ТЕОРЕТИКО-ПРОЕКТНАЯ ЧАСТЬ

2.1 Общая архитектура серверной части

ICC реализован как монолитное серверное приложение Laravel с REST API. Один сервер обрабатывает аккаунты, наборы, конфигурации и публикации. Для учебного объёма это позволяет выполнять связанные операции в одной транзакции MySQL без распределённой синхронизации. Внешний потребитель обращается по HTTP; источником данных служат MySQL и приватное файловое хранилище. Логическая схема приведена на рисунке 2.1.

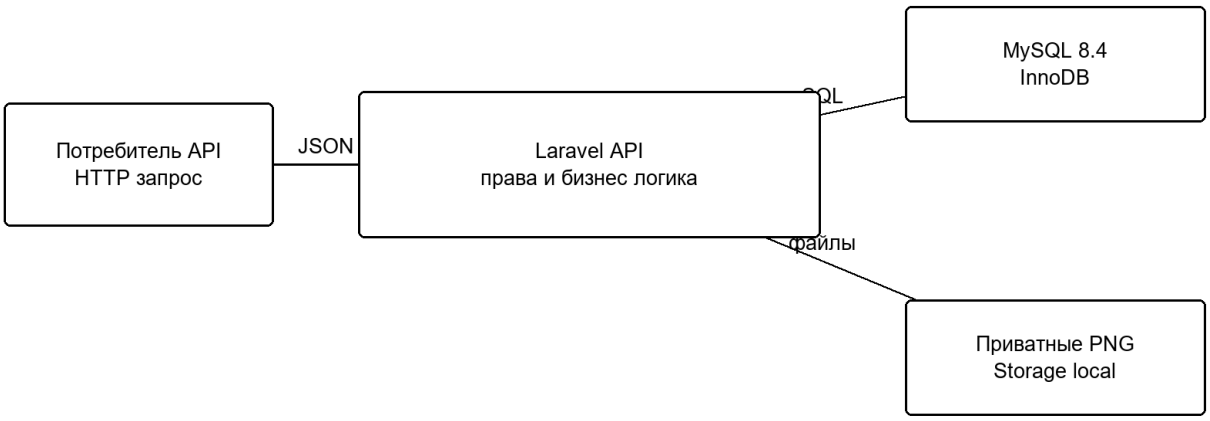


Рисунок 2.1 Логическая архитектура ICC

MySQL хранит владельцев, статусы, пути и связи, но не бинарные PNG. Изображения выдаёт контроллер после проверки политики. Такое разделение обеспечивает НФТ4 и не раскрывает путь storage/app/private как публичный каталог. Потребитель не устанавливает user_id владельца из тела запроса: сервер получает его из контекста Sanctum.

Отдельные сервисы нужны не для сетевого разделения, а для размещения правил. Контроллер отвечает за HTTP, сервис — за согласованное действие, Eloquent — за получение и сохранение модели. Собственный слой репозиториев и DTO в текущем проекте не вводится: для восьми предметных таблиц это дублировало бы построитель запросов без дополнительной задачи.

2.2 Выбор и обоснование технологий

PHP выбран в соответствии с техническим заданием и средой OSPanel. Laravel предоставляет маршрутизацию, контейнер зависимостей, Form Request, API Resource и проверку политик. Эти механизмы непосредственно соответствуют НФТ1–НФТ3.

Сопоставление с Symfony приведено в таблице 2.1; выбор не основывается на недоказанном превосходстве по скорости [9, 18].

Таблица 2.1 Сопоставление PHP фреймворков

Критерий	**Laravel**	**Symfony**
Организация API	Готовые соглашения маршрутов и ресурсов	Гибкая настройка компонентов и контроллеров
Доступ к данным	Eloquent включён в типовой стек	Обычно Doctrine, отдельная интеграция
Безопасность	Sanctum, middleware, Policy	Security, firewalls, voters
Валидация	Form Request и правила	Validator и constraints
Применимость к ICC	Совпадает с этапами заданий	Возможен, но меняет требуемые механизмы

MySQL 8.4 с InnoDB поддерживает внешние ключи, уникальные ограничения, транзакции и блокирующие чтения. PostgreSQL также пригоден, но замена СУБД не даёт ICC необходимой функции и усложняет требуемый запуск. SQLite удобен для отдельных локальных приложений, однако в данном проекте не используется: проверка конкурентного изменения и среда пользователя основаны на отдельном сервере MySQL [14].

Таблица 2.2 Итоговый серверный стек

Средство	**Версия стенда**	**Назначение**
PHP	8.4.19	Исполнение серверной логики и GD
Laravel	13.40.0	Маршруты, контейнер, валидация, тестирование
Eloquent	В составе Laravel	Связи моделей и построение SQL
Sanctum	4.x	Персональные Bearer-токены
MySQL	8.4.8	Реляционные данные и InnoDB
Scribe	5.11.0	HTML, OpenAPI, Postman
OSPanel	6.5.1	PHP, Apache и MySQL в Windows
VS Code	Редактор проекта	Исходники, терминал и задачи

Composer фиксирует зависимости в composer.lock, поэтому повторная установка выполняется командой composer install. Зависимости vendor не включаются в Git. Scribe находится в require-dev, что подходит для генерации документации в учебной среде; при установке без dev-зависимостей используются уже сохранённые статические файлы документации [17].

2.3 Проектирование базы данных

2.3.1 Сущности и связи

Концептуальная модель сохранена из КП 2: User, Maker, Asset, Character, Post, Comment и Like. Промежуточная asset_character связывает персонажа с выбранными элементами. У Maker есть две ссылки на User: автор и последний проверяющий. У Post ссылка на Character необязательна, поскольку допускается загрузка готового PNG без сохранённой конфигурации [5].

Диаграмма связей приведена на рисунке 2.2. Обозначение 1:N означает один объект слева и произвольное число связанных записей справа; M:N реализуется через явную промежуточную таблицу. Обязательность и поведение удаления уточняются в физической модели и словаре полей.

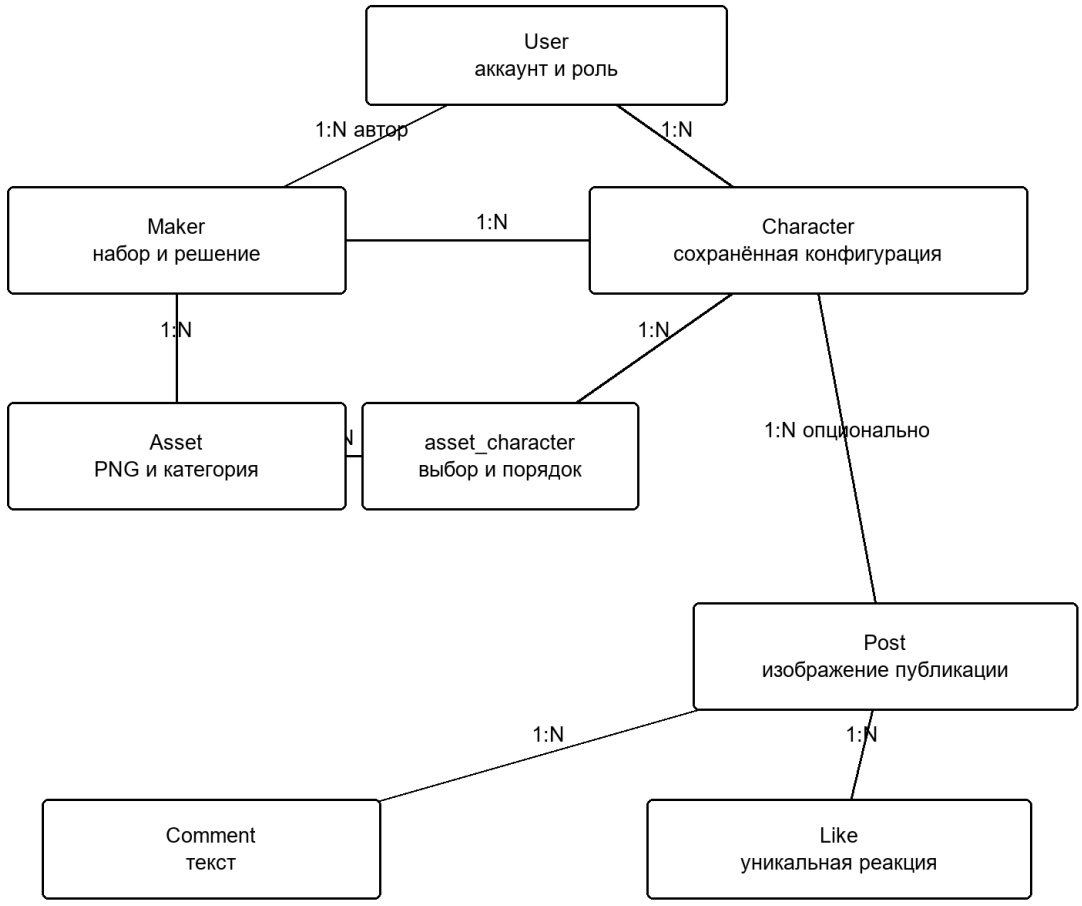


Рисунок 2.2 Предметные сущности и связи ICC

Все Comment и Like дополнительно связаны с User отношением 1:N; Post также имеет владельца User. Эти связи реализованы внешними ключами. Вспомогательный layer_order в asset_character сохраняет порядок конкретной конфигурации и не заменяет исходный порядок

Asset. В результате один элемент может использоваться разными персонажами без дублирования файла.

2.3.2 Нормализация и ограничения

Первая нормальная форма обеспечивается атомарными значениями полей и отдельной строкой для каждой связи выбранного элемента. Список asset_ids не хранится строкой или JSON в Character. Во второй нормальной форме атрибут layer_order промежуточной таблицы относится ко всей паре asset_id и character_id. Имя и категория элемента находятся в Asset и не зависят от части составного ключа pivot.

Третья нормальная форма соблюдается разнесением владельца, набора, элемента и публикации. Имя автора не копируется в Maker или Post, а извлекается по внешнему ключу. review_comment относится к текущему решению Maker, а не к пользовательскому профилю. Исходная модель КП 2 уже была нормализована; физическая реализация уточняет типы, значения по умолчанию, ключи и индексы.

UNIQUE users.email исключает повтор аккаунта с тем же email. UNIQUE likes(user_id, post_id) защищает правило одного лайка независимо от повторных запросов. Составной PK asset_character(asset_id, character_id) исключает повтор элемента в одном персонаже. Перечисления ролей, статусов и категорий хранятся как VARCHAR и проверяются приложением через Enum или whitelist; SQL CHECK для этих перечней в текущей схеме не объявлен.

Таблица 2.3 Политики удаления

Связь	**Действие**	**Обоснование**
User → Maker Character Post	RESTRICT	Не терять владельца сохраняемых объектов
User reviewer → Maker	SET NULL	Сохранить набор после удаления проверяющего
Maker → Asset	CASCADE	Элемент не существует вне набора
Maker → Character	RESTRICT	Не разрушать сохранённую конфигурацию
Character → Post	SET NULL	Изображение публикации сохраняет самостоятельный смысл
Asset Character → pivot	CASCADE	Удалить зависимые строки выбора
Post → Comment Like	CASCADE	Реакции не существуют без публикации

Запрет удаления используемого Asset дополнительно проверяет MakerService. Таким образом, допустимая в SQL каскадная очистка pivot не используется для молчаливого

изменения сохранённого образа через API. Ограничения БД и прикладные правила работают совместно.

2.3.3 Словарь физической модели

В таблицах 2.4–2.11 приведена фактическая структура предметных таблиц MySQL. PK — первичный ключ, FK — внешний ключ, NN — NOT NULL, UQ — уникальность. TIMESTAMP у стандартных created_at и updated_at допускает NULL согласно миграциям Laravel. В MySQL BOOLEAN представлен TINYINT(1). Служебные таблицы рассмотрены отдельно.

Таблица 2.4 Структура users

Поле	Тип MySQL	Ограничения	Назначение
id	BIGINT UNSIGNED	PK; NN	Идентификатор
name	VARCHAR(255)	NN	Название или имя
email	VARCHAR(255)	NN; UNIQUE	Адрес входа
email_verified_at	TIMESTAMP	NULL	Время подтверждения email
password	VARCHAR(255)	NN	Хеш пароля
remember_token	VARCHAR(100)	NULL	Стандартный токен Laravel
created_at	TIMESTAMP	NULL	Создание записи
updated_at	TIMESTAMP	NULL	Изменение записи
role	VARCHAR(20)	NN; DEFAULT 'user'	Роль user creator admin
is_blocked	BOOLEAN	NN; DEFAULT '0'	Блокировка аккаунта

Таблица 2.5 Структура makers

Поле	Тип MySQL	Ограничения	Назначение
id	BIGINT UNSIGNED	PK; NN	Идентификатор
user_id	BIGINT UNSIGNED	NN; FK → users.id; ON DELETE RESTRICT	Владелец или автор
reviewed_by	BIGINT UNSIGNED	NULL; FK → users.id; ON DELETE SET NULL	Последний проверяющий
title	VARCHAR(255)	NN	Название
description	TEXT	NN	Описание набора
status	VARCHAR(20)	NN; DEFAULT 'draft'	Текущий статус
review_comment	TEXT	NULL	Последняя причина решения
submitted_at	DATETIME	NULL	Дата отправки
reviewed_at	DATETIME	NULL	Дата решения
created_at	TIMESTAMP	NULL	Создание записи
updated_at	TIMESTAMP	NULL	Изменение записи

status принимает draft, pending, approved, revision, rejected или hidden. Поля reviewed_by, review_comment и reviewed_at описывают последнее решение, submitted_at — последнюю отправку. Аудит нескольких решений не заменяется этой записью.

Таблица 2.6 Структура assets

Поле	Тип MySQL	Ограничения	Назначение
id	BIGINT UNSIGNED	PK; NN	Идентификатор
maker_id	BIGINT UNSIGNED	NN; FK → makers.id; ON DELETE CASCADE	Связанный мейкер
name	VARCHAR(255)	NN	Название или имя
category	VARCHAR(30)	NN	Категория слоя
file_path	VARCHAR(255)	NN	Приватный путь PNG
layer_order	INT UNSIGNED	NN; DEFAULT '0'	Порядок слоя
is_active	BOOLEAN	NN; DEFAULT '1'	Доступность элемента
created_at	TIMESTAMP	NULL	Создание записи
updated_at	TIMESTAMP	NULL	Изменение записи

Таблица 2.7 Структура characters

Поле	Тип MySQL	Ограничения	Назначение
id	BIGINT UNSIGNED	PK; NN	Идентификатор
user_id	BIGINT UNSIGNED	NN; FK → users.id; ON DELETE RESTRICT	Владелец или автор
maker_id	BIGINT UNSIGNED	NN; FK → makers.id; ON DELETE RESTRICT	Связанный мейкер
name	VARCHAR(255)	NN	Название или имя
created_at	TIMESTAMP	NULL	Создание записи
updated_at	TIMESTAMP	NULL	Изменение записи

Таблица 2.8 Структура asset_character

Поле	Тип MySQL	Ограничения	Назначение
asset_id	BIGINT UNSIGNED	PK; NN; FK → assets.id; ON DELETE CASCADE	Выбранный элемент
character_id	BIGINT UNSIGNED	PK; NN; FK → characters.id; ON DELETE CASCADE	Связанный персонаж
layer_order	INT UNSIGNED	NN; DEFAULT '0'	Порядок слоя
created_at	TIMESTAMP	NULL	Создание записи
updated_at	TIMESTAMP	NULL	Изменение записи

Составной ключ состоит из asset_id и character_id; оба FK удаляют зависимую строку каскадно. Порядок выбранных asset_ids преобразуется сервисом в layer_order от нуля. Уникальность категории среди выбранных элементов проверяется до sync().

Таблица 2.9 Структура posts

Поле	Тип MySQL	Ограничения	Назначение
id	BIGINT UNSIGNED	PK; NN	Идентификатор
user_id	BIGINT UNSIGNED	NN; FK → users.id; ON DELETE RESTRICT	Владелец или автор
character_id	BIGINT UNSIGNED	NULL; FK → characters.id; ON DELETE SET NULL	Связанный персонаж
title	VARCHAR(255)	NN	Название
image_path	VARCHAR(255)	NN	Приватный путь публикации
is_visible	BOOLEAN	NN; DEFAULT '1'	Публичная видимость
created_at	TIMESTAMP	NULL	Создание записи
updated_at	TIMESTAMP	NULL	Изменение записи

Таблица 2.10 Структура comments

Поле	Тип MySQL	Ограничения	Назначение
id	BIGINT UNSIGNED	PK; NN	Идентификатор
user_id	BIGINT UNSIGNED	NN; FK → users.id; ON DELETE CASCADE	Владелец или автор
post_id	BIGINT UNSIGNED	NN; FK → posts.id; ON DELETE CASCADE	Связанная публикация
body	TEXT	NN	Текст комментария
created_at	TIMESTAMP	NULL	Создание записи
updated_at	TIMESTAMP	NULL	Изменение записи

Таблица 2.11 Структура likes

Поле	Тип MySQL	Ограничения	Назначение
id	BIGINT UNSIGNED	PK; NN	Идентификатор
user_id	BIGINT UNSIGNED	NN; FK → users.id; ON DELETE CASCADE; UNIQUE(user_id,post_id)	Владелец или автор
post_id	BIGINT UNSIGNED	NN; FK → posts.id; ON DELETE CASCADE; UNIQUE(user_id,post_id)	Связанная публикация
created_at	TIMESTAMP	NULL	Создание записи
updated_at	TIMESTAMP	NULL	Изменение записи

Для likes дополнительно объявлен UNIQUE(user_id, post_id). Повторный PUT не создаёт вторую реакцию. Каскадное удаление пользователя или публикации очищает зависимые строки.

Служебные таблицы: migrations регистрирует выполненные миграции; password_reset_tokens и sessions относятся к стандартной инфраструктуре Laravel; personal_access_tokens хранит Sanctum-токены. Последняя содержит id, tokenable_type и tokenable_id, name, уникальный token длиной 64, abilities, last_used_at, expires_at и timestamps. Тело Bearer-токена не возвращается при чтении профиля.

2.3.4 Индексы и транзакции

Индексы создаются под реальные запросы каталога и связанных списков. Они ускоряют выборку ценой места и обслуживания при записи, поэтому индекс на каждое поле не требуется. Назначение основных индексов показано в таблице 2.12. Физическая модель с типами также сохранена в docs/physical-model.png и DBML; SQL для независимой проверки находится в sql/schema.mysql.sql.

Таблица 2.12 Индексы и запросы

Индекс	**Назначение**
makers(status)	Публичная выдача approved и список модерации
makers(user_id, status)	Собственные мейкеры по состоянию
makers(title)	Упорядочение и поиск; LIKE с ведущим % требует сканирования
assets(category)	Фильтр по категории
assets(maker_id, layer_order)	Чтение слоёв выбранного набора
posts(is_visible, created_at)	Публичная лента по времени
comments(post_id, created_at)	Комментарии публикации по дате
likes(user_id, post_id) UNIQUE	Единственность реакции и поиск пары

Изменение состояния мейкера выполняется в DB::transaction(). После lockForUpdate() сервис повторно читает актуальную строку и принимает решение внутри блокировки. Сохранение Character и синхронизация asset_character также входят в одну транзакцию. Исключение до завершения приводит к откату, а не к частично сохранённому выбору. Блокирующее чтение InnoDB применяется для защиты конкурентного изменения, а не для блокировки всех чтений каталога [14].

2.4 Слои приложения и ключевые сценарии

Маршрут выбирает контроллер и middleware. Form Request проверяет форму данных до вызова метода. Контроллер выполняет Gate::authorize там, где нужен объектный доступ, передаёт validated() сервису и возвращает API Resource. Модель и связи Eloquent используются внутри сервиса. Схема обработки приведена на рисунке 2.3 [12, 7, 8, 10].

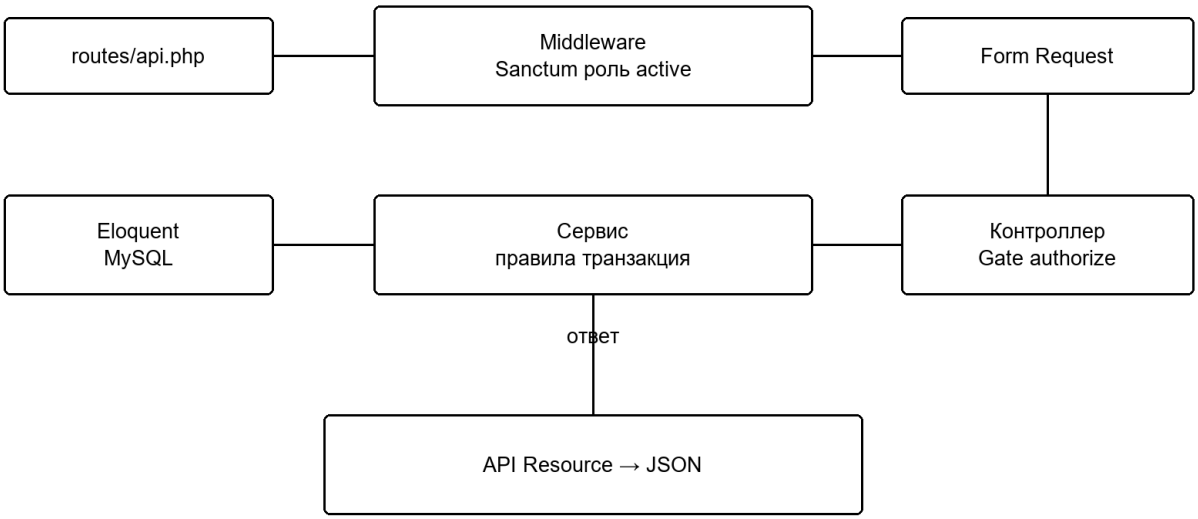


Рисунок 2.3 Обработка запроса Laravel

Ключевой сценарий — отправка и рассмотрение Maker. Креатор создаёт draft, добавляет активные body и face одинакового размера и отправляет заявку. Сервис блокирует строку, проверяет canSubmit и полноту набора, записывает pending и submitted_at. Администратор рассматривает только pending. Для approve повторно проверяется полнота; для request-revision и reject валидация требует объяснение. Переходы приведены в таблице 2.13.

Таблица 2.13 Переходы статусов Maker

Исходный статус	**Действие**	**Результат**
draft или revision	submit после проверки	pending
pending	approve администратором	approved
pending	request-revision с причиной	revision
pending	reject с причиной	rejected
approved	Правка владельцем или admin	draft и очистка решения
approved	hide администратором	hidden
draft или revision	Удаление без Character	Запись удалена
Иной недопустимый переход	Повтор submit review или hide	409 без изменения

hidden и rejected не разрешают редактирование или повторную отправку в текущем Enum. Это не переименование revision: revision означает разрешённую доработку, hidden —

административное снятие, rejected — отказ. Сервер явно сохраняет эти различия. Листинг сервиса в приложении 3 позволяет проверить условия по коду.

Второй сценарий — сохранение персонажа. После авторизации сервер блокирует Maker, проверяет approved, получает активные элементы только из этого набора и сравнивает их число с запросом. Затем проверяет отсутствие повторных категорий. Лишь после этого создаётся Character и выполняется sync() порядка в pivot. Ошибка 422 не оставляет пустой персонаж; статус неодобренного Maker приводит к 409.

Третий сценарий — публикация и реакции. PNG проходит отдельную проверку и сохраняется в приватное хранилище. character_id, если передан, должен принадлежать автору. Комментарий привязывается к доступной публикации. PUT like обеспечивает идемпотентное создание одной пары, DELETE like снимает её. Административное скрытие исключает публикацию из публичной ленты и закрывает публичную выдачу изображения.

2.5 Проектирование API

Базовый префикс текущего API — /api. Версия в URL не выделена. Методы GET читают, POST создают ресурс или выполняют доменное действие, PUT и PATCH изменяют, DELETE удаляет. Для списка используется data с links и meta, для одной сущности — data. Удаление возвращает 204 без тела. Исключения health и входа описаны отдельно. Полная таблица маршрутов вынесена в приложение 1.

Таблица 2.14 Основные группы API

Группа	**Операции**	**Доступ**
Авторизация	register login user logout	Публичный вход, профиль и выход с токеном
Мейкеры	Список карточка CRUD submit	Публичное чтение approved; creator и admin
Элементы	Добавление правка удаление PNG	Владелец набора и admin
Персонажи	CRUD и порядок выбора	Авторизованный владелец и admin
Публикации	Список карточка создание удаление	Публичные и собственные данные по Policy
Реакции	Комментарий лайк снятие удаление	Авторизованные с проверкой объекта
Администрирование	review hide users statistics	Только admin

Общие ошибки: 401 означает отсутствие допустимой аутентификации, 403 — отсутствие права или блокировку, 404 — отсутствующий либо недоступный по правилам публичной выдачи ресурс, 409 — конфликт текущего состояния, 422 — неверные поля. Ограничение частоты возвращает 429. Непредвиденная ошибка при APP_DEBUG=false даёт безопасное message с кодом 500. Проверка доступности БД может вернуть 503.

Листинг 2.1 Пример запроса сохранения конфигурации

```
POST /api/characters
Accept: application/json
Authorization: Bearer <token>
Content-Type: application/json

{"name":"Мой герой","maker_id":1,"asset_ids":[1,2,3]}
```

В примере токен заменён условным обозначением. Сервер определяет user_id по аутентифицированному пользователю. Порядок asset_ids сохраняется как selected_layer_order в ответе ресурса. Несуществующий ID обнаруживается Form Request, а существующий элемент другого набора — сервисом.

Листинг 2.2 Единый формат ошибки полей

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/json
{
  "message": "Ошибка валидации",
  "errors": {"asset_ids": ["Все элементы должны быть активны и принадлежать одному
мейкеру."]}
}
```

Контракт зафиксирован в routes/api.php, Form Request, Resource, docs/endpoints.csv и генерируемой спецификации OpenAPI. При интеграции используются именно эти поля и коды. Дополнительный фильтр, не реализованный в сервисе конкретного списка, не считается поддерживаемой операцией этого списка.

2.6 Проектирование механизмов безопасности

Sanctum работает в режиме персональных Bearer-токенов. При успешном входе токен выдаётся клиенту один раз, а в personal_access_tokens хранится его SHA-256 хеш. Срок действия конфигурации — 1440 минут. Отдельного refresh-токена в проекте нет: после истечения срока требуется новый вход. Выход удаляет текущий токен; блокировка или смена роли удаляет все токены пользователя [11].

Пароль хранится через cast hashed и проверяется Hash::check. Он исключён из UserResource и скрытых полей модели. Правила регистрации запрещают передавать role и is_blocked: самостоятельная регистрация всегда создаёт user. Назначение creator выполняет admin. Администратор не может заблокировать себя или изменить собственную роль через этот API [15].

Таблица 2.15 Угрозы и серверные меры

Угроза	**Механизм ICC**	**Проверка**
Чужой идентификатор	Policy и проверка родительского Maker	403 либо 404
Повышение роли	Запрет role в регистрации, role middleware	422 и 403
SQL-инъекция	Параметры Query Builder, whitelist сортировки	Недопустимый sort даёт 422
Вредоносный файл	MIME PNG, GD декодирование и повторная запись	422 для поддельного файла
Повторные запросы	UNIQUE пары Like и проверка статуса	Один лайк, 409 для submit
Перебор входа	Лимит 10 запросов в минуту по IP	429 после лимита
Утечка конфигурации	public как web-root, .env вне Git	В ответе нет SQL и пароля
Чужой origin	CORS whitelist без credentials	Нет ACAO для чужого origin

CORS ограничивает чтение ответов браузером и не заменяет серверную авторизацию. Для разрешённых origin доступны Authorization и Content-Type. API работает с JSON; экранирование отображаемого текста остаётся обязанностью любого потребителя данных. Приватный PNG выдаётся с Content-Type image/png, nosniff и запретом публичного кэширования. Декодирование и повторная запись удаляют посторонние вложенные данные, но не подменяют смысловую модерацию рисунка.

2.7 Контроль версий и выводы по разделу

Исходники размещены в публичном репозитории https://github.com/Rat-mir/ICC. Основная ветка main содержит отдельные этапы КП 3–12, последующую настройку MySQL и материалы проверки. Названия коммитов связывают изменение с заданием. Даты отражают фактическое сохранение в Git; они не используются как доказательство выполнения до сроков Classroom. Соответствие этапов представлено в подразделе 3.1 и docs/commit-map.md [3].

В .gitignore исключены .env, .env.testing, vendor, node_modules, приватные PNG и рабочие журналы. Для передачи проекта используется .env.example без реального секрета. Напарник может клонировать main и создать ветку для своей задачи. История выполненных pull request и code review не заявляется: опубликована существующая локальная последовательность коммитов.

Архитектура обеспечивает целостность MySQL, доступ через Sanctum и Policy, валидацию Form Request и представление Resource. Модель сохраняет семь сущностей КП 2 и отдельную связь выбора слоёв.

3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И ОЦЕНКА РЕЗУЛЬТАТОВ

3.1 Среда разработки и история коммитов

Рабочая папка проекта — C:\OSPanel\home\ICC. Серверные модули запускаются средствами OSPanel; редактирование, Git и команды Artisan выполняются в VS Code. PHP 8.4 используется по полному пути, если он не выбран в окружении терминала. Версии компонентов стенда указаны в таблице 2.2. Установка на другом компьютере не требует переносить vendor или рабочие пароли.

Этапы разработки зафиксированы отдельными коммитами. Таблица 3.1 позволяет открыть изменение каждого задания. Сначала был создан каркас Laravel, затем добавлена физическая модель; порядок фактической истории сохранён, а номера этапов указаны в сообщениях. Последующая настройка MySQL переводит текущий запуск и тесты полностью на эту СУБД.

Таблица 3.1 Коммиты серверных этапов

Этап	Коммит	Содержание
КП 3	0886f85	Физическая модель SQL и JOIN
КП 4	5ce4b77	Каркас Laravel Sanctum окружение
КП 5	6efd33b	Миграции модели фабрики Requests Resources
КП 6	bf21e80	Маршруты контроллеры и сервисы
КП 7	7045ff4	CRUD связанные данные и ошибки
КП 8	b0f4864	Токены роли политики и журнал
КП 9	1ee3231	Модерация транзакции фильтры статистика
КП 10	f4bcc7a	Тесты тест-кейсы и лог
MySQL	14fa582	Рабочая и тестовая MySQL OSPanel VS Code
Проверка	02b0f6c	Подтверждение MySQL HTTPS и сценариев

На рисунке 3.1 показан фрагмент опубликованной истории с серверными этапами КП 3–10. Эти же коммиты доступны локально в VS Code; ссылка на конкретное изменение строится по его хешу.

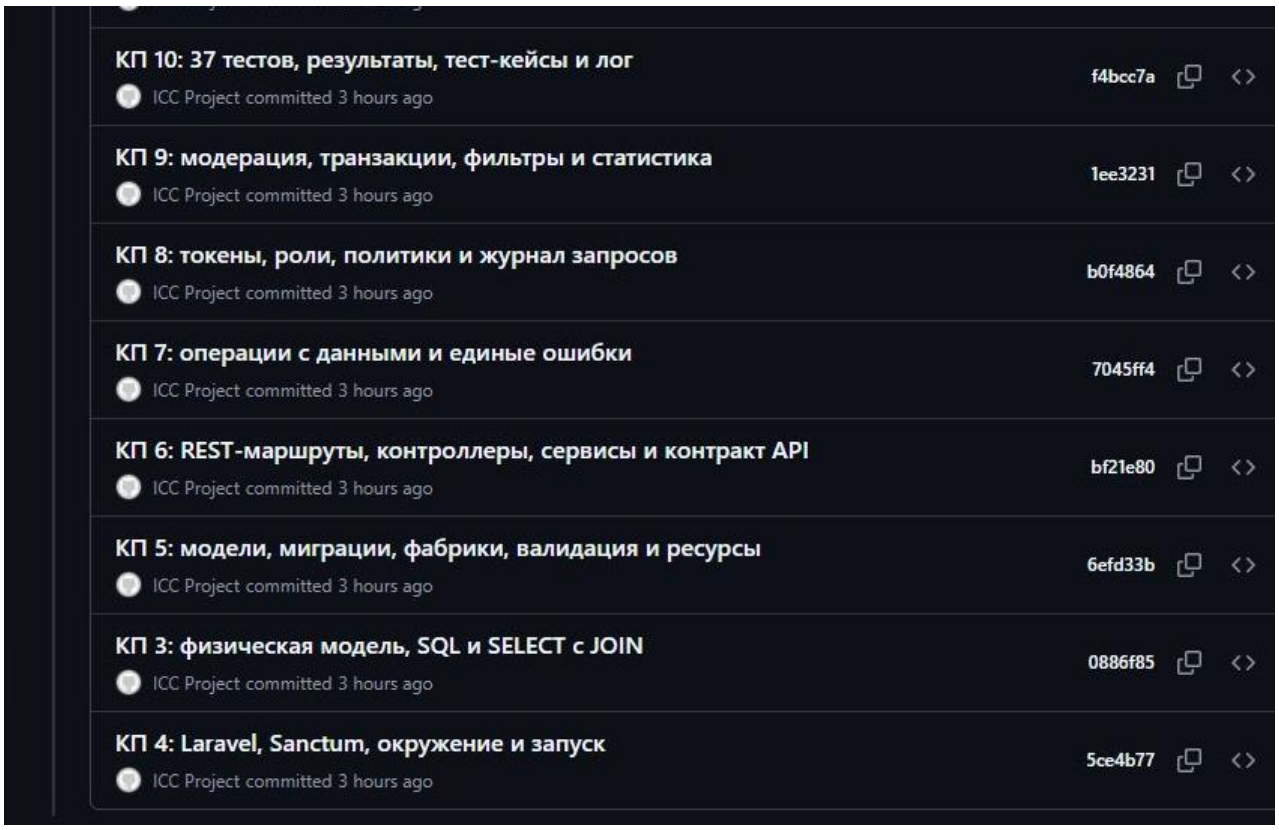


Рисунок 3.1 Коммиты КП 3–10 в публичном репозитории

Файлы проекта организованы по ответственности. Дерево основных серверных каталогов показано в листинге 3.1. Сгенерированная документация public/docs доступна без установки Node.js. Исходные технические документы хранятся в docs/sources; повторяемые результаты — в docs и tests.

Листинг 3.1 Структура серверного проекта

```
app/
  Enums/              роли статусы категории
  Exceptions/         BusinessException
  Http/Controllers/Api/  HTTP операции
  Http/Middleware/    доступ лимиты журнал
  Http/Requests/      проверка входных данных
  Http/Resources/     представление JSON
  Models/             Eloquent и связи
  Policies/           права на объекты
  Services/           сценарии и файлы
bootstrap/app.php     маршрутизация middleware ошибки
config/               mysql sanctum cors logging
database/             migrations factories seeders
routes/api.php        контракт маршрутов
tests/                Feature и Unit
sql/                  независимые DDL данные JOIN
public/docs/          HTML OpenAPI Postman
tools/                настройка и проверка MySQL
```

3.2 Реализация базы данных и начальное наполнение

Миграции создают стандартные users и таблицы Laravel, personal_access_tokens, отдельным шагом добавляют role и is_blocked, затем создают предметные таблицы в порядке зависимостей. Для обычной установки используется migrate --seed. Полный код миграций приведён в приложении 2. Фрагмент создания Maker показан в листинге 3.2 [7].

Листинг 3.2 Основные поля миграции makers

```
Schema::create('makers', function (Blueprint $t) {
    $t->id();
    $t->foreignId('user_id')->constrained()->restrictOnDelete();
    $t->foreignId('reviewed_by')->nullable()
      ->constrained('users')->nullOnDelete();
    $t->string('title');
    $t->text('description');
    $t->string('status', 20)->default('draft');
    $t->text('review_comment')->nullable();
    $t->dateTime('submitted_at')->nullable();
    $t->dateTime('reviewed_at')->nullable();
    $t->timestamps();
    $t->index('status');
    $t->index('title');
    $t->index(['user_id', 'status']);
});
```

Пользователь задаётся внешним ключом с RESTRICT, проверяющий допускает NULL. Это различает постоянное владение объектом и необязательные сведения о последнем решении. В миграции нет поля с массивом элементов: связь конфигурации создаётся отдельной таблицей. Значение draft устанавливается БД по умолчанию и согласовано с Enum.

Модели используют $fillable, скрытые пароль и remember_token, casts boolean, даты и Enum. Связи belongsTo связывают автора и родительский объект, hasMany возвращает зависимые записи, belongsToMany связывает Character и Asset. Для выбранного порядка используется withPivot(layer_order). Предварительная загрузка with() и load() предотвращает последовательное чтение каждого автора или слоя отдельным запросом [9].

Таблица 3.2 Eloquent связи моделей

Модель	**Связи**
User	hasMany makers characters posts comments likes
Maker	belongsTo creator reviewer; hasMany assets characters
Asset	belongsTo maker; belongsToMany characters
Character	belongsTo user maker; belongsToMany assets
Post	belongsTo user character; hasMany comments likes
Comment и Like	belongsTo user post

DatabaseSeeder создаёт три известных аккаунта ролей и ещё семь пользователей, 10 мейкеров разных состояний и по шесть PNG-категорий на набор. Для 10 персонажей создаются строки pivot, публикации, комментарии и лайки. Фабрики формируют демонстрационные данные, генератор DemoArtwork создаёт изображения. После сидирования каждый основной тип имеет не менее 10 строк, что превышает минимум задания.

Схема дополнительно проверена независимым выполнением DDL и INSERT в icc_backend_ddl. Пять SELECT с JOIN проверяют автора набора, элементы, состав персонажа, публикации и статистику. Фактические результаты сохранены в docs/mysql-ddl-check.json и docs/join-results.md. Это позволяет сопоставить миграции с реляционной моделью, а не ограничиваться просмотром исходного PHP.

3.3 Маршруты валидация ответы и ошибки

routes/api.php группирует публичное чтение, загрузку файлов, защищённые операции, роль creator/admin и административный префикс. apiResource используется для ресурсных действий; submit, review и hide выделены как доменные операции. В таблице маршрутов 37 записей; одна запись обновления может принимать PUT и PATCH. В приложении 1 они указаны как альтернативные методы одного действия.

Листинг 3.3 Защищённая группа креатора

```
Route::middleware('role:creator,admin')->group(function () {
    Route::apiResource('makers', MakerController::class)
        ->only(['store', 'update', 'destroy']);
    Route::post('makers/{maker}/submit',
        [MakerController::class, 'submit']);
    Route::post('makers/{maker}/assets',
        [AssetController::class, 'store']);
});
```

Группа находится внутри auth:sanctum, active и throttle:api. Роль не проверяет владельца — это задача Policy. Вложенные маршруты Asset дополнительно проверяют совпадение maker_id с родительским объектом, поэтому нельзя изменить элемент через идентификатор другого набора.

Form Request отделяет проверку структуры от правил состояния. CharacterRequest требует имя, существующий maker_id и непустой массив asset_ids с distinct и exists. Это ещё не доказывает совместимость: принадлежность одному набору и уникальность категорий проверяет CharacterService. Такая граница позволяет возвращать ошибку конкретного поля и сохранять доменные проверки рядом с транзакцией [12].

Листинг 3.4 Правила запроса персонажа

```
return [
    'name' => 'required|string|max:255',
    'maker_id' => 'required|integer|exists:makers,id',
    'asset_ids' => 'required|array|min:1|max:30',
    'asset_ids.*' => 'required|integer|distinct|exists:assets,id',
];
```

Контроллер передаёт сервису только validated(). При создании персонажа ответ устанавливается в 201; для show и update используется 200; для удаления — 204. Resource задаёт набор публичных полей и подключает связанные данные только при загрузке. UserResource не возвращает пароль, remember_token или хеш Sanctum [8].

Листинг 3.5 Метод создания CharacterController

```
public function store(CharacterRequest $r) {
    return (new CharacterResource(
        $this->service->save($r->user(), $r->validated())
    ))->response()->setStatusCode(201);
}
```

Обработчики исключений находятся в bootstrap/app.php. ValidationException формирует message и errors с 422. BusinessException даёт 409 с объяснением. Конфликт ссылочной или уникальной целостности преобразуется в 409, если его не перехватил предварительный валидатор. Отсутствующая запись возвращает 404. При выключенном debug непредвиденное исключение не раскрывает SQL и стек; подробности сохраняются в серверном журнале.

Таблица 3.3 Ответы при нарушении правил

Проверка	**Результат**	**Состояние данных**
Нет токена для /api/characters	401 message	Изменений нет
user создаёт Maker	403 message	Maker не создан
Элемент другого родителя	404 message	Asset не изменён
Повтор submit для pending	409 message	Статус pending сохранён
Отказ без review_comment	422 errors	Решение не записано
Поддельный PNG или 8×8	422 errors.image	Файл не принят
Чужой character_id в Post	403 message	Публикация не создана

3.4 Аутентификация авторизация и файлы

При регистрации пароль получает hashed-cast модели User. При входе AuthService ищет email и вызывает Hash::check. Сообщение для неверного email и пароля одинаково, чтобы

не сообщать существование аккаунта. Блокированный пользователь получает 403. Возвращаемый объект содержит user, token, token_type=Bearer и expires_in в секундах. Используется именно персональный токен Sanctum, а не JWT и не cookie-сеанс [11, 15].

Листинг 3.6 Выдача токена Sanctum

```
public function token(User $user): array {
    return [
        'user' => new UserResource($user),
        'token' => $user->createToken('icc-api')->plainTextToken,
        'token_type' => 'Bearer',
        'expires_in' => config('sanctum.expiration') * 60,
    ];
}
```

EnsureNotBlocked исключает использование защищённых маршрутов заблокированным аккаунтом. EnsureUserHasRole проверяет нужную роль; Gate и Policy проверяют конкретную запись. Например, MakerPolicy допускает просмотр approved всем, а приватный набор — активному владельцу и admin. Изменение требует creator/admin и владения или административного права. Эти проверки не зависят от видимости элемента управления у потребителя.

Листинг 3.7 Объектная проверка изменения мейкера

```
public function update(User $user, Maker $maker): bool {
    return $this->create($user)
        && ($user->isAdmin() || $user->id === $maker->user_id);
}
```

AssetRequest проверяет настоящий PNG, MIME, ограничение 5120 КБ и стороны от 16 до 2048. ImageService декодирует его средствами GD и повторно записывает в PNG с сохранением альфа-канала. Имя заменяется UUID. Файл хранится на диске local в приватном каталоге, в БД сохраняется относительный путь. Публичная символическая ссылка storage:link не требуется и не используется для этих изображений.

При ошибке записи строки после сохранения нового PNG сервис удаляет созданный файл. Удаление набора сначала проверяет ссылки и транзакционно удаляет строки, затем очищает файлы. При замене Asset старые PNG сейчас не очищаются автоматически; это ограничение хранения, а не механизм версий. Отдельная фоновая уборка и версионирование отнесены к дальнейшему развитию.

3.5 Бизнес логика транзакции и списки

В MakerService сосредоточены операции списка, изменения, работы с PNG, отправки, рассмотрения и скрытия. Проверка complete() требует активные body и face и равные размеры всех активных PNG. canSubmit разрешает draft и revision. Перед сменой состояния строка

повторно загружается под lockForUpdate(). Фрагмент приведён в листинге 3.8, полный модуль — в приложении 3.

Листинг 3.8 Транзакционная отправка на модерацию

```
return DB::transaction(function () use ($maker) {
    $maker = Maker::lockForUpdate()->findOrFail($maker->id);
    if (!$maker->status->canSubmit()) {
        throw new BusinessException(
            'Отправка разрешена только из draft или revision.'
        );
    }
    $this->complete($maker);
    $maker->status = MakerStatus::Pending;
    $maker->submitted_at = now();
    $maker->reviewed_by = null;
    $maker->reviewed_at = null;
    $maker->review_comment = null;
    $maker->save();
    return $maker;
});
```

Проверка полноты расположена до save(). При отсутствующем face или разных размерах сохраняются draft и пустой submitted_at. Если approved-набор редактируется, editable() переводит его в draft и очищает прошлое решение в той же транзакции. Каталог больше не выдаёт такой набор до нового одобрения. Это закрывает возможность подмены проверенного содержимого.

CharacterService сохраняет Character и pivot атомарно. При повторных категориях или элементе чужого Maker не создаётся ни основная запись, ни строки выбора. Удаление используемого мейкера или элемента приводит к BusinessException 409. Проверка двух соединений MySQL дополнительно подтвердила блокировку конкурентного изменения и сохранность данных после rollback.

Параметры каталога обрабатывает CatalogRequest: search до 100 символов, category из шести значений, status через Enum, диапазон дат, sort из id/title/created_at/updated_at, order asc/desc, page от 1 и per_page от 1 до 100. Сервис последовательно применяет when(), search(), orderBy() и paginate()->withQueryString(). По умолчанию выдаётся 20 записей. Публичная область сначала ограничивается approved; переданный status не открывает приватные строки.

Листинг 3.9 Фрагмент запроса списка мейкеров

```
return $q->search($f['search'] ?? null)
    ->when($f['status'] ?? null,
        fn ($q, $v) => $q->where('status', $v))
    ->when($f['category'] ?? null,
        fn ($q, $v) => $q->whereHas('assets',
            fn ($q) => $q->where('category', $v)
                         ->where('is_active', true)))
    ->orderBy($f['sort'] ?? 'created_at', $f['order'] ?? 'desc')
    ->orderBy('id')
    ->paginate($f['per_page'] ?? 20)->withQueryString();
```

Дополнительная сортировка по id делает порядок определённым при одинаковом основном значении. В тесте из пяти совпадений страница 2 с размером 2 возвращает две строки, total=5 и last_page=3. sort=password и per_page=999 дают 422. Административная статистика использует groupBy(status) и count(), а не подсчёт строк потребителем API.

3.6 Журналирование и документирование API

ApiLogger фиксирует метод, путь без query string, user_id, код ответа и длительность обработки. Тело запроса, Authorization и пароль не записываются. MakerService добавляет события maker.submitted и maker.reviewed уровня info, а попытку некорректного рассмотрения — warning. Непредвиденные исключения обрабатываются штатным журналом Laravel. Доступ к реальным журналам остаётся локальным; пример для отчёта обезличен.

Scribe формирует HTML, OpenAPI YAML и Postman-коллекцию по PHPDoc контроллеров. В документации указываются параметр, роль, пример ответа и типовые ошибки. Команда php artisan scribe:generate обновляет public/docs. Статическая HTML-страница открывается на /docs/ у API-origin. OpenAPI описывает контракт, а коллекция используется для повторения сценария [17].

В текущем контракте запросы с изображением используют multipart/form-data. Для правки метаданных допустим JSON; обновление файла через клиент PHP может передаваться POST с _method=PUT. В JSON даты представлены строками, отсутствующие связи допускают null. Потребитель обязан обрабатывать 204 без попытки прочитать пустое тело JSON. Для междоменного обращения разрешены только origin из CLIENT_ORIGINS.

При проверке интеграции API использовались два HTTPS-origin OSPanel. Проверялись Authorization Bearer, OPTIONS и отсутствие доступа у неверной роли. Настройка CORS не использует wildcard и supports_credentials=false. Эта часть описывает серверный контракт взаимодействия; реализация пользовательских страниц в записку не включается.

3.7 Тестирование серверной части

Проверка выполнялась на отдельной MySQL icc_backend_test с RefreshDatabase и фабриками. Рабочая icc_backend не используется PHPUnit. Повторный прогон от 12 ноября 2026 года содержит 37 тестов и 178 assertions, без errors и failures: 33 прикладных Feature-сценария ICC, три Unit-проверки перечисления и один стандартный smoke-тест Laravel. Процент покрытия кода не измерялся [10].

Таблица 3.4 содержит протокол основных серверных сценариев. Полные исходные данные и фактические результаты всех 33 прикладных кейсов находятся в docs/test-cases.csv. Это функциональные проверки с обращением к БД, а не перечень планируемых действий.

Таблица 3.4 Протокол тестирования

№	Проверка	Условия	Ожидается	Получено
1	Проверка MySQL health	Без тела	200; database=ok	200; database=ok
2	Регистрация и защита роли	name/email/password/confirmation; затем role=admin	201 и роль user; с role=admin —422; пароль хеширован	201 и роль user; с role=admin —422; пароль хеширован
3	Уникальный email и подтверждение	Существующий email; неверное подтверждение	422; errors.email и errors.password	422; errors.email и errors.password
4	Вход профиль выход отзыв	Валидный пароль; затем выданный Bearer	200 вход;200 user;204 выход;401 после отзыва	200 вход;200 user;204 выход;401 после отзыва
5	Нет токена или неверный пароль	Неверный пароль; нет токена; неверный токен	401 для всех случаев	401 для всех случаев
6	Блокировка пользователя	is_blocked=true	403 для входа и защищённого запроса	403 для входа и защищённого запроса
7	Публичный каталог	В БД approved/pending/hidden	200; ровно1 approved; meta.total=1	200; ровно1 approved; meta.total=1
8	Роль создания мейкера	title,description; user и creator	user403; creator201; status=draft и верный user_id	user403; creator201; status=draft и верный user_id

Продолжение таблицы 3.4

№	Проверка	Условия	Ожидается	Получено
9	Владелец и отсутствующий ID	Чужой мейкер; отсутствующий ID	403;404 с message	403;404 с message
10	Собственные черновики	Два черновика разных владельцев	200; только1 свой	200; только1 свой
11	Загрузка PNG и порядок	name/category/body/layer_order; PNG64×64	201; затем200 и layer_order=20	201; затем200 и layer_order=20
12	Тип размер и содержимое PNG	JPEG; PNG8×8; ложный PNG; файл5121КБ	422; errors.image	422; errors.image
13	Чужой родитель элемента	Asset другого мейкера	404; изменение не выполнено	404; изменение не выполнено
14	Полнота набора и откат	Только слой body, без face	422; status=draft, submitted_at=null	422; status=draft, submitted_at=null
15	Равенство размеров PNG	body64×64 и face128×128	422; errors.assets	422; errors.assets
16	Возврат доработка одобрение	draft→pending→revision →pending→approved; причина обязательна	200 на переходах;409 повтор submit/правка pending;422 без причины	200 на переходах;409 повтор submit/правка pending;422 без причины

Продолжение таблицы 3.4

№	Проверка	Условия	Ожидается	Получено
17	Права и повторное решение	creator; затем admin с reject и повтор approve	403;200 rejected;409 повторное решение	403;200 rejected;409 повторное решение
18	Редактирование approved	Изменение описания approved	200; status=draft; в каталоге0 записей	200; status=draft; в каталоге0 записей
19	Приватный PNG	Гость; draft, затем approved	403 для draft;200 image/png для approved	403 для draft;200 image/png для approved
20	CRUD персонажа и порядок	name/maker_id/asset_ids с перестановкой	201/200/200/204; pivot сохраняет порядок и удаляется	201/200/200/204; pivot сохраняет порядок и удаляется
21	Чужие и повторные категории	Asset чужого мейкера; два body	422; нет новых characters и pivot	422; нет новых characters и pivot
22	Неодобренный набор	Неодобренный maker	409	409
23	Владение персонажем	Токен чужого владельца	403;403; пустой свой список	403;403; пустой свой список
24	Публикация комментарий лайк	PNG64×64,title,body; повтор PUT like	201;201; один лайк при повторе; снятие0;204 удаления	201;201; один лайк при повторе; снятие0;204 удаления

Продолжение таблицы 3.4

№	Проверка	Условия	Ожидается	Получено
25	Чужой персонаж в публикации	Чужой character_id; корректный PNG	403	403
26	Комментарий и владение	Пустой body; чужие объекты	422 body;403 для чужих удалений	422 body;403 для чужих удалений
27	Скрытая публикация	is_visible=false, гость	Пустая лента;404 карточка и файл	Пустая лента;404 карточка и файл
28	Скрытие статистика аккаунты	Admin; роль creator; блокировка; самоблокировка	200 hidden;200 false; верная статистика;200 изменения;409 себе	200 hidden;200 false; верная статистика;200 изменения;409 себе
29	Фильтры и meta	5 совпадений; затем sort=password/per_page=999	200;2 строки,total5,page2,last_page3;422 плохие фильтры	200;2 строки,total5,page2,last_page3;422 плохие фильтры
30	Удаление используемых данных	Есть сохранённый Character с выбранным Asset	409 в обоих случаях; строки сохранены	409 в обоих случаях; строки сохранены
31	CORS preflight	Origin127.0.0.1:5173, затем untrusted.example	204; ACAO только для разрешённого origin	204; ACAO только для разрешённого origin
32	Безопасная ошибка	APP_DEBUG=false	404; нет trace и exception в JSON	404; нет trace и exception в JSON

Продолжение таблицы 3.4

№	Проверка	Условия	Ожидается	Получено
33	Уникальность лайка в MySQL	Повторная пара user_id/post_id	QueryException; UNIQUE защищает целостность	QueryException; UNIQUE защищает целостность

Все перечисленные случаи завершились ожидаемым результатом. Проверки подтверждают как положительные операции, так и отсутствие изменений после отказа. Для ФТ8–ФТ11 существенны повтор submit, отказ без причины, возврат revision, повторная отправка и снятие approved после правки. Для НФТ2 важны владение и блокировка, а не только успешный вход.

На рисунке 3.2 приведено сохранённое представление JUnit-протокола первоначального успешного прогона. Повторная проверка 12 ноября дала те же 37 тестов и 178 assertions без ошибок. Исходный XML находится в docs/test-results.xml.

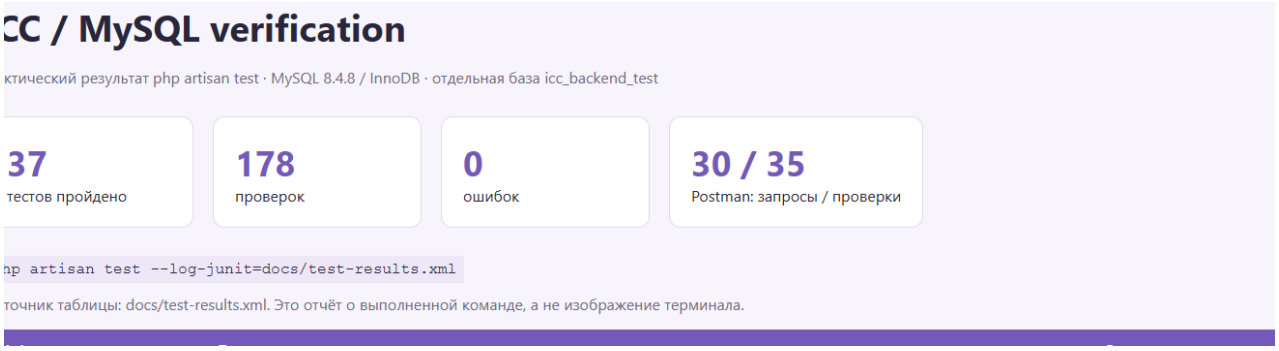


Рисунок 3.2 Представление результатов серверных тестов

Дополнительно выполнены 30 реальных HTTP-запросов воспроизводимой Postman-коллекции через runner Newman: 35 проверок, ошибок нет. В сценарии присутствуют 401 без токена, 403 для креатора на review, 409 при повторной отправке, 422 без причины, затем возврат, правка, повторная отправка, одобрение, сохранение персонажа, публикация и реакции. Результаты находятся в docs/postman-results.json и docs/postman-results.md. Это автоматический запуск коллекции; ручной прогон мышью в приложении Postman не заявляется.

Для транзакций выполнена независимая проверка двумя PDO-соединениями. Первое удерживало блокировку выбранной строки InnoDB, второе пыталось её изменить и получило ошибку ожидания 1205. После rollback исходные данные сохранились. Файл docs/mysql-lock-check.json фиксирует blocked_concurrent_update=true и rollback_preserved_data=true. Проверка подтверждает действие блокировки на MySQL, а не только наличие lockForUpdate в коде.

Контроль HTTPS health на домене OSPanel вернул database=ok, driver=mysql и server_version=8.4.8. Прикладной сценарий также проверен на этом стенде. Время отдельного

прогона тестов не интерпретируется как нагрузочная характеристика: сетевые задержки, одновременные посетители и длительное насыщение сервера не измерялись.

3.8 Развёртывание в OSPanel и оценка результатов

Проект рассчитан на OSPanel и VS Code. На новом компьютере следует клонировать репозиторий в C:\OSPanel\home\ICC, включить PHP-8.4 и MySQL-8.4 и открыть папку в VS Code. Далее выполнить команды листинга 3.10 из терминала с выбранным PHP OSPanel. Скрипт setup-mysql.php создаёт только схемы ICC и локальную конфигурацию, не удаляя другие базы.

Листинг 3.10 Установка и запуск через Artisan

```
cd C:\OSPanel\home\ICC
composer install
php tools/setup-mysql.php
php artisan migrate --seed
php artisan optimize:clear
php artisan serve --host=127.0.0.1 --port=8000
```

Если команда php не найдена или указывает на другую версию, перед artisan указывается полный путь: & "C:\OSPanel\modules\PHP-8.4\php.exe" artisan serve --host=127.0.0.1 --port=8000. Проверка сервера: http://127.0.0.1:8000/api/health; документация: http://127.0.0.1:8000/docs/. Этот Artisan-запуск использует HTTP для локальной разработки. Для проверки требования HTTPS используется настроенный Apache OSPanel и адрес https://api.icc.local/api/health.

Хост сервера Artisan и хост MySQL имеют разное назначение. Для HTTP выбран 127.0.0.1 и порт 8000; для MySQL данного OSPanel — DB_HOST=127.0.1.31, DB_PORT=3306. Подставлять 3306 в artisan serve не нужно. При иной конфигурации MySQL адрес берётся из выбранного модуля OSPanel и меняется в локальной .env. APP_URL должен совпадать с используемым API-origin, затем выполняется optimize:clear.

Таблица 3.5 Основные переменные окружения

Переменная	**Значение или назначение**
APP_NAME / APP_LOCALE	ICC / ru
APP_DEBUG	false для проверки безопасных ошибок
APP_URL	http://127.0.0.1:8000 либо https://api.icc.local
DB_CONNECTION	mysql
DB_HOST / DB_PORT	127.0.1.31 / 3306 на данном стенде
DB_DATABASE	icc_backend; тесты используют icc_backend_test
DB_USERNAME / DB_PASSWORD	Локальные учётные данные, не публикуются
CLIENT_ORIGINS	Явный список разрешённых origin
SANCTUM_EXPIRATION	1440 минут

Миграции применяются командой migrate. Команда migrate:fresh --seed удаляет таблицы выбранной БД и допустима только для пересоздания собственной демонстрационной среды. Для обычного запуска существующего проекта она не требуется. Архив данных должен включать и SQL-дамп, и storage/app/private: одних путей в БД недостаточно для восстановления изображений.

Листинг 3.11 Проверка установленного сервера

```
php artisan route:list --path=api
php artisan test
php artisan scribe:generate
php tools/verify-mysql.php
php tools/verify-lock.php
```

Демонстрационные email: admin@icc.test, creator@icc.test и user@icc.test; общий пароль DemoPass123! предназначен только для учебных аккаунтов. После передачи напарнику код устанавливается через composer.lock, а локальные пароли и APP_KEY создаются заново. Для обычной ручной настройки можно скопировать .env.example, заполнить MySQL и выполнить php artisan key:generate перед миграциями.

Таблица 3.6 связывает требования с фактическим доказательством. Для краткости несколько требований объединены только там, где они проверяются одним сценарием. Файлы доказательств находятся в репозитории, что позволяет повторить проверку и сопоставить отчёт с исходниками.

Таблица 3.6 Требования и доказательства

Требование	**Проверка**	**Реализация или файл**
ФТ1	Кейсы 7 19 27	MakerService PostService Policy
ФТ2	Кейсы 2–6	AuthService RegisterRequest Sanctum
ФТ3	Кейс 29	CatalogRequest и MakerService
ФТ4	Кейсы 20–23	CharacterService и pivot
ФТ5 ФТ6	Кейсы 24–27 33	PostService InteractionController UNIQUE
ФТ7	Кейсы 8–13	MakerService ImageService
ФТ8 ФТ9 ФТ11	Кейсы 14–18	Enum ReviewRequest транзакции
ФТ10 ФТ12	Кейс 28	AdminUserController statistics hide
НФТ1	health и DDL проверка	ospanel-check.json mysql-ddl-check.json
НФТ2 НФТ3	Кейсы 2–6 9 17 32	Requests Policy bootstrap/app.php
НФТ4	Кейсы 11 12 15	AssetRequest GD приватный Storage
НФТ5	Кейс 29	meta per_page links
НФТ6	Два PDO соединения 1205 и rollback	mysql-lock-check.json
НФТ7	Журнал запросов и событий	ApiLogger api-log.txt
НФТ8	HTTPS health и кейс 31	ospanel-check.json config/cors.php
НФТ9	37 тестов 178 assertions	test-results.xml public/docs

Сервер реализует заявленную предметную модель, 37 маршрутов API и полный цикл модерации. Данные хранятся в MySQL 8.4; конфигурации рабочей и тестовой БД разделены. Сохранившиеся ограничения: нет истории всех решений, автоматической уборки заменённых PNG, отдельного refresh-токена, измерения покрытия и нагрузочного протокола.

Перспективы развития — журнал переходов отдельной таблицей, версии набора для стабильности ранее сохранённых конфигураций, фоновая уборка неиспользуемых файлов, полнотекстовый поиск и проверка резервного восстановления. Для производственного запуска дополнительно потребуются отдельные учётные данные, политика хранения и нагрузочные испытания.

ЗАКЛЮЧЕНИЕ

В результате курсового проекта разработана серверная часть ICC на PHP и Laravel с MySQL 8.4. Цель достигнута: API обеспечивает хранение наборов и персонажей, публикации и реакции, а также контролируемый сценарий модерации с разграничением доступа.

По первой задаче проанализирован модельный процесс, сопоставлены три аналога и сформулированы 12 функциональных и 9 нефункциональных серверных требований. Основной проблемой определено отсутствие единого контроля состояния набора, совместимости и владения данными.

По второй задаче выбрана архитектура монолитного API с отдельными сервисами, спроектированы реляционные связи и контракт маршрутов. Семь основных сущностей и промежуточная таблица сохраняют модель КП 2. В физической схеме определены внешние ключи, уникальность и индексы.

По третьей задаче реализованы миграции, модели Eloquent, фабрики и DatabaseSeeder. Начальное наполнение содержит не менее 10 строк в каждой основной сущности; выбранные элементы вынесены в pivot. Независимое выполнение SQL и JOIN подтвердило работоспособность модели MySQL.

По четвёртой задаче реализованы маршруты и контроллеры, Form Request, API Resource и общие JSON-ошибки. По пятой задаче настроены Sanctum, роли, Policy, блокировка аккаунтов, приватное PNG-хранилище и журнал без паролей и токенов.

По шестой задаче реализованы допустимые переходы модерации, обязательная причина возврата и отказа, снятие approved после правки, транзакции и lockForUpdate. Каталог поддерживает поиск, фильтры, сортировку, пагинацию; административный API возвращает агрегированную статистику.

По седьмой задаче выполнены 37 тестов с 178 проверками, 30 запросов Postman-коллекции с 35 проверками и независимый контроль блокировки двумя MySQL-соединениями. Подготовлены документация API, инструкция OSPanel и запуск Artisan. Исходники и этапные коммиты опубликованы в https://github.com/Rat-mir/ICC.

Представленный вклад охватывает серверную модель, бизнес-логику, API, безопасность и проверку. Направления дальнейшего развития определены в подразделе 3.8: история решений, версии наборов, уборка файлов и нагрузочное тестирование. Они расширяют текущий результат, не изменяя подтверждённых функций учебного сервера.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ASTANA POLYTECHNIC. Рекомендации по написанию разделов пояснительной записки курсового проекта по дисциплине Основы backend-разработки. 2026. Classroom, приложение к ПР 13.

2. CHARAT. Avatar Maker [Электронный ресурс]. URL: https://charat.me/ru/index.php (дата обращения: 12.11.2026).

3. Git Documentation. git log [Электронный ресурс]. URL: https://git-scm.com/docs/git-log (дата обращения: 12.11.2026).

4. ICC Исходный код и результаты проверки [Электронный ресурс]. URL: https://github.com/Rat-mir/ICC (дата обращения: 12.11.2026).

5. ICC КП 2 Проектирование структуры базы данных. Недосекова А. И., Ибрагимов Р. Р. Файл ICC_КП2.docx. Копия: docs/sources репозитория проекта.

6. ICC Техническое задание. Недосекова А. И., Ибрагимов Р. Р. Группа ИС-24-22б. Файл ICC_Техническое_задание_ICC.docx. Копия: docs/sources репозитория проекта.

7. Laravel 13. Database Migrations [Электронный ресурс]. URL: https://laravel.com/docs/13.x/migrations (дата обращения: 12.11.2026).

8. Laravel 13. Eloquent API Resources [Электронный ресурс]. URL: https://laravel.com/docs/13.x/eloquent-resources (дата обращения: 12.11.2026).

9. Laravel 13. Eloquent Getting Started [Электронный ресурс]. URL: https://laravel.com/docs/13.x/eloquent (дата обращения: 12.11.2026).

10. Laravel 13. HTTP Tests [Электронный ресурс]. URL: https://laravel.com/docs/13.x/http-tests (дата обращения: 12.11.2026).

11. Laravel 13. Sanctum [Электронный ресурс]. URL: https://laravel.com/docs/13.x/sanctum (дата обращения: 12.11.2026).

12. Laravel 13. Validation [Электронный ресурс]. URL: https://laravel.com/docs/13.x/validation (дата обращения: 12.11.2026).

13. meiker.io. Free Game Creation Tool [Электронный ресурс]. URL: https://meiker.io/ (дата обращения: 12.11.2026).

14. MySQL 8.4 Reference Manual. Locking Reads [Электронный ресурс]. URL: https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html (дата обращения: 12.11.2026).

15. PHP Manual. password_hash [Электронный ресурс]. URL: https://www.php.net/manual/en/function.password-hash.php (дата обращения: 12.11.2026).

16. Picrew Support. Creator functions [Электронный ресурс]. URL: https://support.picrew.me/en/category/creator_functions (дата обращения: 12.11.2026).

17. Scribe. Introduction [Электронный ресурс]. URL: https://scribe.knuckles.wtf/laravel/ (дата обращения: 12.11.2026).

18. Symfony Documentation. Security [Электронный ресурс]. URL: https://symfony.com/doc/current/security.html (дата обращения: 12.11.2026).

ПРИЛОЖЕНИЕ 1 СПЕЦИФИКАЦИЯ API

Префикс /api включён в маршруты. P — публичная операция с ограничением видимости, U — активный авторизованный пользователь, C — creator или admin с проверкой владельца, A — admin. Для изменения ресурса PUT и PATCH являются альтернативами. Общие ошибки 429 и 500 применимы к защищённым операциям, ограничения видимости и состояния уточняются Policy и сервисом.

Таблица П1.1 Маршруты API

Метод	URL	Роль	Назначение	Ответы
GET	/api/admin/makers	A	Список для модерации	200 / 401 403 404 409 422
POST	/api/admin/makers/{maker}/hide	A	Скрытие	200 / 401 403 404 409 422
POST	/api/admin/makers/{maker}/review	A	Решение администратора	200 / 401 403 404 409 422
GET	/api/admin/posts	A	Список для модерации	200 / 401 403 404 409 422
POST	/api/admin/posts/{post}/hide	A	Скрытие	200 / 401 403 404 409 422
GET	/api/admin/statistics	A	Агрегированная статистика	200 / 401 403 404 409 422
GET	/api/admin/users	A	Список	200 / 401 403 404 409 422
PATCH	/api/admin/users/{user}	A	Изменение	200 / 401 403 404 409 422
GET	/api/assets/{asset}/image	P	PNG с проверкой доступа	200 / 401 403 404 409 422
GET	/api/characters	U	Список	200 / 401 403 404 409 422

Продолжение таблицы П1.1

Метод	**URL**	**Роль**	**Назначение**	**Ответы**
POST	/api/characters	U	Создание	201 / 401 403 404 409 422
GET	/api/characters/{character}	U	Карточка	200 / 401 403 404 409 422
PUT PATCH	/api/characters/{character}	U	Изменение	200 / 401 403 404 409 422
DELETE	/api/characters/{character}	U	Удаление	204 / 401 403 404 409 422
DELETE	/api/comments/{comment}	U	Удаление	204 / 401 403 404 409 422
GET	/api/health	P	Проверка приложения и БД	200 / 503
POST	/api/login	P	Вход и токен	200 / 401 403 422 429
POST	/api/logout	U	Отзыв токена	204 / 401 403 404 409 422
GET	/api/makers	P	Список	200 / 401 403 404 409 422
POST	/api/makers	C	Создание	201 / 401 403 404 409 422

Продолжение таблицы П1.1

Метод	URL	Роль	Назначение	Ответы
GET	/api/makers/{maker}	P	Карточка	200 / 401 403 404 409 422
PUT PATCH	/api/makers/{maker}	C	Изменение	200 / 401 403 404 409 422
DELETE	/api/makers/{maker}	C	Удаление	204 / 401 403 404 409 422
POST	/api/makers/{maker}/assets	C	Создание	201 / 401 403 404 409 422
PUT PATCH	/api/makers/{maker}/assets/{asset}	C	Изменение	200 / 401 403 404 409 422
DELETE	/api/makers/{maker}/assets/{asset}	C	Удаление	204 / 401 403 404 409 422
POST	/api/makers/{maker}/submit	C	Отправить на модерацию	200 / 401 403 404 409 422
GET	/api/posts	P	Список	200 / 401 403 404 409 422
POST	/api/posts	U	Создание	201 / 401 403 404 409 422
GET	/api/posts/{post}	P	Карточка	200 / 401 403 404 409 422

Продолжение таблицы П1.1

Метод	URL	Роль	Назначение	Ответы
DELETE	/api/posts/{post}	U	Удаление	204 / 401 403 404 409 422
POST	/api/posts/{post}/comments	U	Добавление комментария	201 / 401 403 404 409 422
GET	/api/posts/{post}/image	P	PNG с проверкой доступа	200 / 401 403 404 409 422
PUT	/api/posts/{post}/like	U	Поставить лайк	200 / 401 403 404 409 422
DELETE	/api/posts/{post}/like	U	Снять лайк	200 / 401 403 404 409 422
POST	/api/register	P	Регистрация	201 / 422 429
GET	/api/user	U	Текущий пользователь	200 / 401 403 404 409 422

Полные параметры и примеры JSON: public/docs/openapi.yaml и docs/API.md. Коллекция: docs/ICC.postman_collection.json. Сценарный прогон: ICC.smoke.postman_collection.json. Доступ к PNG возвращает image/png, а не JSON-ресурс.

ПРИЛОЖЕНИЕ 2 МИГРАЦИИ БАЗЫ ДАННЫХ

Листинги соответствуют файлам database/migrations. Порядок определяется именами файлов; стандартные users создаются до внешних ключей. Комментарии и код приведены без локальных паролей.

Листинг П2 0001_01_01_000000_create_users_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('password');
            $table->rememberToken();
            $table->timestamps();
        });

        Schema::create('password_reset_tokens', function (Blueprint $table) {
            $table->string('email')->primary();
            $table->string('token');
            $table->timestamp('created_at')->nullable();
        });

        Schema::create('sessions', function (Blueprint $table) {
            $table->string('id')->primary();
            $table->foreignId('user_id')->nullable()->index();
            $table->string('ip_address', 45)->nullable();
            $table->text('user_agent')->nullable();
            $table->longText('payload');
            $table->integer('last_activity')->index();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('users');
        Schema::dropIfExists('password_reset_tokens');
        Schema::dropIfExists('sessions');
    }
};
```

Листинг П2 2026_11_06_182749_create_personal_access_tokens_table.php

```
<?php
```

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('personal_access_tokens', function (Blueprint $table) {
            $table->id();
            $table->morphs('tokenable');
            $table->text('name');
            $table->string('token', 64)->unique();
            $table->text('abilities')->nullable();
            $table->timestamp('last_used_at')->nullable();
            $table->timestamp('expires_at')->nullable()->index();
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('personal_access_tokens');
    }
};
```

Листинг П2 2026_11_11_000001_add_roles_to_users.php

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
return new class extends Migration {
    public function up(): void { Schema::table('users', function(Blueprint $t) {
$t->string('role',20)->default('user'); $t->boolean('is_blocked')->default(false); }); }
    public function down(): void { Schema::table('users',fn(Blueprint
$t)=>$t->dropColumn(['role','is_blocked'])); }
};
```

Листинг П2 2026_11_11_000002_create_icc_tables.php

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
return new class extends Migration {
    public function up(): void {
        Schema::create('makers',function(Blueprint $t) {
            $t->id(); $t->foreignId('user_id')->constrained()->restrictOnDelete();
            $t->foreignId('reviewed_by')->nullable()->constrained('users')->nullOnDelete();
            $t->string('title'); $t->text('description');
$t->string('status',20)->default('draft');
            $t->text('review_comment')->nullable(); $t->dateTime('submitted_at')->nullable();
 $t->dateTime('reviewed_at')->nullable(); $t->timestamps();
            $t->index('status'); $t->index('title'); $t->index(['user_id','status']);
        });
        Schema::create('assets',function(Blueprint $t) {
            $t->id(); $t->foreignId('maker_id')->constrained()->cascadeOnDelete();
            $t->string('name'); $t->string('category',30); $t->string('file_path');
```

```
$t->unsignedInteger('layer_order')->default(0);
            $t->boolean('is_active')->default(true); $t->timestamps(); $t->index('category');
 $t->index(['maker_id','layer_order']);
        });
        Schema::create('characters',function(Blueprint $t) {
            $t->id(); $t->foreignId('user_id')->constrained()->restrictOnDelete();
$t->foreignId('maker_id')->constrained()->restrictOnDelete(); $t->string('name');
$t->timestamps();
        });
        Schema::create('asset_character',function(Blueprint $t) {
            $t->foreignId('asset_id')->constrained()->cascadeOnDelete();
$t->foreignId('character_id')->constrained()->cascadeOnDelete();
            $t->unsignedInteger('layer_order')->default(0); $t->timestamps();
$t->primary(['asset_id','character_id']);
        });
        Schema::create('posts',function(Blueprint $t) {
            $t->id(); $t->foreignId('user_id')->constrained()->restrictOnDelete();
            $t->foreignId('character_id')->nullable()->constrained()->nullOnDelete();
            $t->string('title'); $t->string('image_path');
$t->boolean('is_visible')->default(true); $t->timestamps();
$t->index(['is_visible','created_at']);
        });
        Schema::create('comments',function(Blueprint $t) {
            $t->id(); $t->foreignId('user_id')->constrained()->cascadeOnDelete();
$t->foreignId('post_id')->constrained()->cascadeOnDelete(); $t->text('body');
$t->timestamps(); $t->index(['post_id','created_at']);
        });
        Schema::create('likes',function(Blueprint $t) {
            $t->id(); $t->foreignId('user_id')->constrained()->cascadeOnDelete();
$t->foreignId('post_id')->constrained()->cascadeOnDelete(); $t->timestamps();
$t->unique(['user_id','post_id']);
        });
    }
    public function down(): void {
foreach(['likes','comments','posts','asset_character','characters','assets','makers'] as
$table) Schema::dropIfExists($table); }
};
```

ПРИЛОЖЕНИЕ 3 ОСНОВНЫЕ СЕРВЕРНЫЕ МОДУЛИ

Маршруты, MakerService и MakerPolicy показывают разделение HTTP, бизнес-правил и объектного доступа. Это полные серверные модули; краткие поясняющие фрагменты разобраны в основной части.

Листинг П3 routes/api.php

```
<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\{AuthController,MakerController,AssetController,CharacterControl
ler,PostController,InteractionController,AdminUserController,HealthController};
Route::get('health',HealthController::class)->name('api.health');
Route::middleware('throttle:login')->group(function() {
    Route::post('register',[AuthController::class,'register'])->name('api.register');
    Route::post('login',[AuthController::class,'login'])->name('api.login');
});
Route::middleware(['optional.auth','throttle:api'])->group(function() {
    Route::apiResource('makers',MakerController::class)->only(['index','show']);
    Route::apiResource('posts',PostController::class)->only(['index','show']);
});
Route::middleware(['optional.auth','throttle:media'])->group(function() {

Route::get('assets/{asset}/image',[AssetController::class,'image'])->name('assets.image');
    Route::get('posts/{post}/image',[PostController::class,'image'])->name('posts.image');
});
Route::middleware(['auth:sanctum','active','throttle:api'])->group(function() {
    Route::get('user',[AuthController::class,'user'])->name('api.user');
    Route::post('logout',[AuthController::class,'logout'])->name('api.logout');
    Route::apiResource('characters',CharacterController::class);
    Route::apiResource('posts',PostController::class)->only(['store','destroy']);
    Route::post('posts/{post}/comments',[InteractionController::class,'comment'])-
>name('comments.store');
    Route::delete('comments/{comment}',[InteractionController::class,'destroy'])-
>name('comments.destroy');

Route::put('posts/{post}/like',[InteractionController::class,'like'])->name('likes.store');
    Route::delete('posts/{post}/like',[InteractionController::class,'unlike'])-
>name('likes.destroy');
    Route::middleware('role:creator,admin')->group(function() {

Route::apiResource('makers',MakerController::class)->only(['store','update','destroy']);
        Route::post('makers/{maker}/submit',[MakerController::class,'submit'])-
>name('makers.submit');

Route::post('makers/{maker}/assets',[AssetController::class,'store'])->name('assets.store');
        Route::match(['put','patch'],'makers/{maker}/assets/{asset}',[AssetController::class,
'update'])->name('assets.update');
        Route::delete('makers/{maker}/assets/{asset}',[AssetController::class,'destroy'])-
>name('assets.destroy');
    });
    Route::prefix('admin')->name('admin.')->middleware('role:admin')->group(function() {
        Route::get('makers',[MakerController::class,'moderation'])->name('makers.index');
        Route::post('makers/{maker}/review',[MakerController::class,'review'])-
>name('makers.review');

Route::post('makers/{maker}/hide',[MakerController::class,'hide'])->name('makers.hide');
        Route::get('posts',[PostController::class,'moderation'])->name('posts.index');
        Route::post('posts/{post}/hide',[PostController::class,'hide'])->name('posts.hide');
        Route::get('statistics',[MakerController::class,'statistics'])->name('statistics');
        Route::get('users',[AdminUserController::class,'index'])->name('users.index');

Route::patch('users/{user}',[AdminUserController::class,'update'])->name('users.update');
```

```
    });
});
```

Листинг П3 app/Services/MakerService.php

```
<?php
namespace App\Services;
use App\Enums\MakerStatus;
use App\Exceptions\BusinessException;
use App\Models\{User,Maker,Asset};
use Illuminate\Support\Facades\{DB,Log,Storage};
use Illuminate\Support\Arr;
use Illuminate\Validation\ValidationException;
class MakerService {
    public function __construct(private ImageService $images) {}
    public function list(array $f,?User $user,bool $admin=false) {
        $q=Maker::with('creator')->withCount(['assets'=>fn($q)=>$q-
>where('is_active',true),'characters']);
        if($admin) { /* Explicit admin route only. */ }
        elseif(!empty($f['mine']) && $user) $q->where('user_id',$user->id);
        else $q->where('status',MakerStatus::Approved->value);
        return $q->search($f['search']??null)
            ->when($f['status']??null,fn($q,$v)=>$q->where('status',$v))
            ->when($f['category']??null,fn($q,$v)=>$q->whereHas('assets',fn($q)=>$q-
>where('category',$v)->where('is_active',true)))
            ->when($f['from']??null,fn($q,$v)=>$q->whereDate('created_at','>=',$v))
            ->when($f['to']??null,fn($q,$v)=>$q->whereDate('created_at','<=',$v))
            ->orderBy($f['sort']??'created_at',$f['order']??'desc')->orderBy('id')
            ->paginate($f['per_page']??20)->withQueryString();
    }
    public function detail(Maker $maker,?User $user): Maker {
        $private=$user && ($user->isAdmin() || $user->id===$maker->user_id);
        return $maker->load(['creator','assets'=>fn($q)=>$q->when(!$private,fn($q)=>$q-
>where('is_active',true))])->loadCount('characters');
    }
    public function create(User $user,array $data): Maker { return
$user->makers()->create($data); }
    public function update(Maker $maker,array $data): Maker {
        return DB::transaction(function()use($maker,$data) {
            $maker=Maker::lockForUpdate()->findOrFail($maker->id); $this->editable($maker);
            $maker->fill($data)->save(); return $maker;
        });
    }
    private function editable(Maker $maker): void {
        if(!$maker->status->canEdit()) throw new BusinessException('Набор можно менять только
 в черновике, после возврата или снятия с публикации для повторной проверки.');
        if($maker->status===MakerStatus::Approved) {
            $maker->status=MakerStatus::Draft; $maker->reviewed_by=null;
$maker->reviewed_at=null; $maker->review_comment=null; $maker->save();
        }
    }
    public function delete(Maker $maker): void {
        $paths=DB::transaction(function()use($maker) {
            $maker=Maker::lockForUpdate()->findOrFail($maker->id);
            if(!in_array($maker->status,[MakerStatus::Draft,MakerStatus::Revision],true))
throw new BusinessException('Удалить можно только черновик или возвращённый набор.');
            if($maker->characters()->exists()) throw new BusinessException('Набор
используется сохранёнными персонажами.');
            $paths=$maker->assets()->pluck('file_path')->all(); $maker->delete(); return
$paths;
        });
        Storage::disk('local')->delete($paths);
    }
    public function asset(Maker $maker,array $data,?Asset $asset=null): Asset {
        $path=isset($data['image'])?$this->images->store($data['image'],'assets'):null;
        try {
```

```
            $saved=DB::transaction(function()use($maker,$data,$asset,$path) {
                $maker=Maker::lockForUpdate()->findOrFail($maker->id);
$this->editable($maker);
                if($asset) { $asset=Asset::lockForUpdate()->findOrFail($asset->id);
abort_unless($asset->maker_id===$maker->id,404); }
                else { $asset=new Asset; $asset->maker()->associate($maker); }
                $asset->fill(Arr::except($data,['image'])); if($path)
$asset->file_path=$path; $asset->save(); return $asset;
            });
        } catch(\Throwable $e) { if($path) Storage::disk('local')->delete($path); throw $e; }
        return $saved;
    }
    public function deleteAsset(Maker $maker,Asset $asset): void {
        $path=DB::transaction(function()use($maker,$asset) {
            $maker=Maker::lockForUpdate()->findOrFail($maker->id); $this->editable($maker);
            $asset=Asset::lockForUpdate()->findOrFail($asset->id);
abort_unless($asset->maker_id===$maker->id,404);
            if($asset->characters()->exists()) throw new BusinessException('Элемент
используется сохранённым персонажем.');
            $path=$asset->file_path; $asset->delete(); return $path;
        });
        Storage::disk('local')->delete($path);
    }
    private function complete(Maker $maker): void {
        $assets=$maker->assets()->where('is_active',true)->get();
        if($assets->isEmpty() || !$assets->contains('category','body') ||
!$assets->contains('category','face'))
            throw ValidationException::withMessages(['assets'=>['Нужны активные слои body и
face.']]);
        $size=null;
        foreach($assets as $asset) {
            $dimensions=$this->images->dimensions($asset->file_path);
            if($size && $size!==$dimensions) throw
ValidationException::withMessages(['assets'=>['Все PNG набора должны иметь одинаковые
размеры.']]);
            $size=$dimensions;
        }
    }
    public function submit(Maker $maker): Maker {
        return DB::transaction(function()use($maker) {
            $maker=Maker::lockForUpdate()->findOrFail($maker->id);
            if(!$maker->status->canSubmit()) throw new BusinessException('Отправка разрешена
только из draft или revision.');
            $this->complete($maker); $maker->status=MakerStatus::Pending;
$maker->submitted_at=now();
            $maker->reviewed_by=null; $maker->reviewed_at=null; $maker->review_comment=null;
$maker->save();
            Log::info('maker.submitted',['maker_id'=>$maker->id,'user_id'=>$maker->user_id]);
 return $maker;
        });
    }
    public function review(Maker $maker,User $admin,array $data): Maker {
        return DB::transaction(function()use($maker,$admin,$data) {
            $maker=Maker::lockForUpdate()->findOrFail($maker->id);
            if($maker->status!==MakerStatus::Pending) {
Log::warning('maker.invalid_review',['maker_id'=>$maker->id]); throw new
BusinessException('Рассматривать можно только заявку pending.'); }
            if($data['decision']==='approve') $this->complete($maker);
            $maker->status=match($data['decision'])
{'approve'=>MakerStatus::Approved,'reject'=>MakerStatus::Rejected,'request-
revision'=>MakerStatus::Revision};
            $maker->reviewed_by=$admin->id; $maker->reviewed_at=now();
$maker->review_comment=$data['review_comment']??null; $maker->save();
            Log::info('maker.reviewed',['maker_id'=>$maker->id,'admin_id'=>$admin-
>id,'decision'=>$data['decision']]); return $maker;
        });
    }
```

```
    public function hide(Maker $maker,User $admin): Maker {
        return DB::transaction(function()use($maker,$admin) {
            $maker=Maker::lockForUpdate()->findOrFail($maker->id);
            if($maker->status!==MakerStatus::Approved) throw new BusinessException('Скрыть
можно только опубликованный мейкер.');
            $maker->status=MakerStatus::Hidden; $maker->reviewed_by=$admin->id;
$maker->reviewed_at=now(); $maker->save(); return $maker;
        });
    }
    public function statistics(): array {
        return ['makers_by_status'=>Maker::selectRaw('status, count(*) as
total')->groupBy('status')->pluck('total','status'),
            'users'=>User::count(),'posts'=>\App\Models\Post::count(),'characters'=>\App\Mode
ls\Character::count()];
    }
}
```

Листинг П3 app/Policies/MakerPolicy.php

```
<?php
namespace App\Policies;
use App\Models\{User,Maker};
use App\Enums\{MakerStatus,UserRole};
class MakerPolicy {
    public function view(?User $user, Maker $maker): bool { return
$maker->status===MakerStatus::Approved || ($user && !$user->is_blocked && ($user->isAdmin()
|| $user->id===$maker->user_id)); }
    public function create(User $user): bool { return
in_array($user->role,[UserRole::Creator,UserRole::Admin],true); }
    public function update(User $user, Maker $maker): bool { return $this->create($user) &&
($user->isAdmin() || $user->id===$maker->user_id); }
    public function delete(User $user, Maker $maker): bool { return
$this->update($user,$maker); }
}
```

ПРИЛОЖЕНИЕ 4 РЕПОЗИТОРИЙ И МАТЕРИАЛЫ ПРОВЕРКИ

Репозиторий: https://github.com/Rat-mir/ICC

История: https://github.com/Rat-mir/ICC/commits/main/

Для просмотра коммита используется адрес https://github.com/Rat-mir/ICC/commit/ и хеш из таблицы 3.1. Проверки MySQL и API находятся в docs; автоматические сценарии — в tests. История сохраняется и в локальной папке C:\OSPanel\home\ICC, доступной в VS Code.

Таблица П4.1 Расположение доказательств

Материал	**Путь в репозитории**
Требования и проверка	docs/compliance.md, docs/verification.md
Физическая модель	docs/physical-model.md, .dbml, .png
SQL и JOIN	sql/schema.mysql.sql, sql/seed.mysql.sql, sql/joins.sql
Прогон PHPUnit	docs/test-results.xml
Все 33 кейса	docs/test-cases.csv
HTTP сценарий	docs/postman-results.json
Блокировка InnoDB	docs/mysql-lock-check.json
OSPanel HTTPS MySQL	docs/ospanel-check.json
Документация	public/docs/index.html, public/docs/openapi.yaml
Развёртывание	README.md, tools/setup-mysql.php